

Data Analytics

Einführung in OLAP, Data Warehouses etc.

Prof. Stefan Keller, FS26

- Online Analytical Processing (OLAP)
- Data Warehouses
- Aggregations-Funktionen in SQL
- Data Marts
- OLAP Cubes
- Data Warehouse, Data Lake, Data Mesh, Data Space
- Materialien:
 - Fallstudie Weinhandlung (Ergänzung)
 - Analytische (Window-) Funktionen in SQL (Repetition)

- Sie kennen die Grundlagen und Anwendungen von Data Warehouses und OLAP
- Sie können einfache relationale Data Warehouses entwerfen
- Sie kennen SQL-Aggregations-Operatoren und können diese anwenden
- Sie kennen die Konzepte und Implementationstechniken von Data Marts
- Sie kennen die Konzepte von OLAP Data Cubes
- Sie kennen Konzepte von Data Lakes, Data Meshes und Data Spaces

EINFÜHRUNG OLAP

- Online Transaction Processing (OLTP)
- Data Warehouse-Anwendungen:
 - Online Analytical Processing (OLAP)
 - vorhandene Informationen analysieren
 - Data Mining
 - herausfinden, welche Informationen es gibt
 - Decision Support Systems
 - Business Intelligence / Datenvisualisierung
 - Machine Learning
 - zukünftige Trends vorhersagen
- Weitere Konzepte im Ausblick

Einführung: Online Transaction Processing (OLTP)

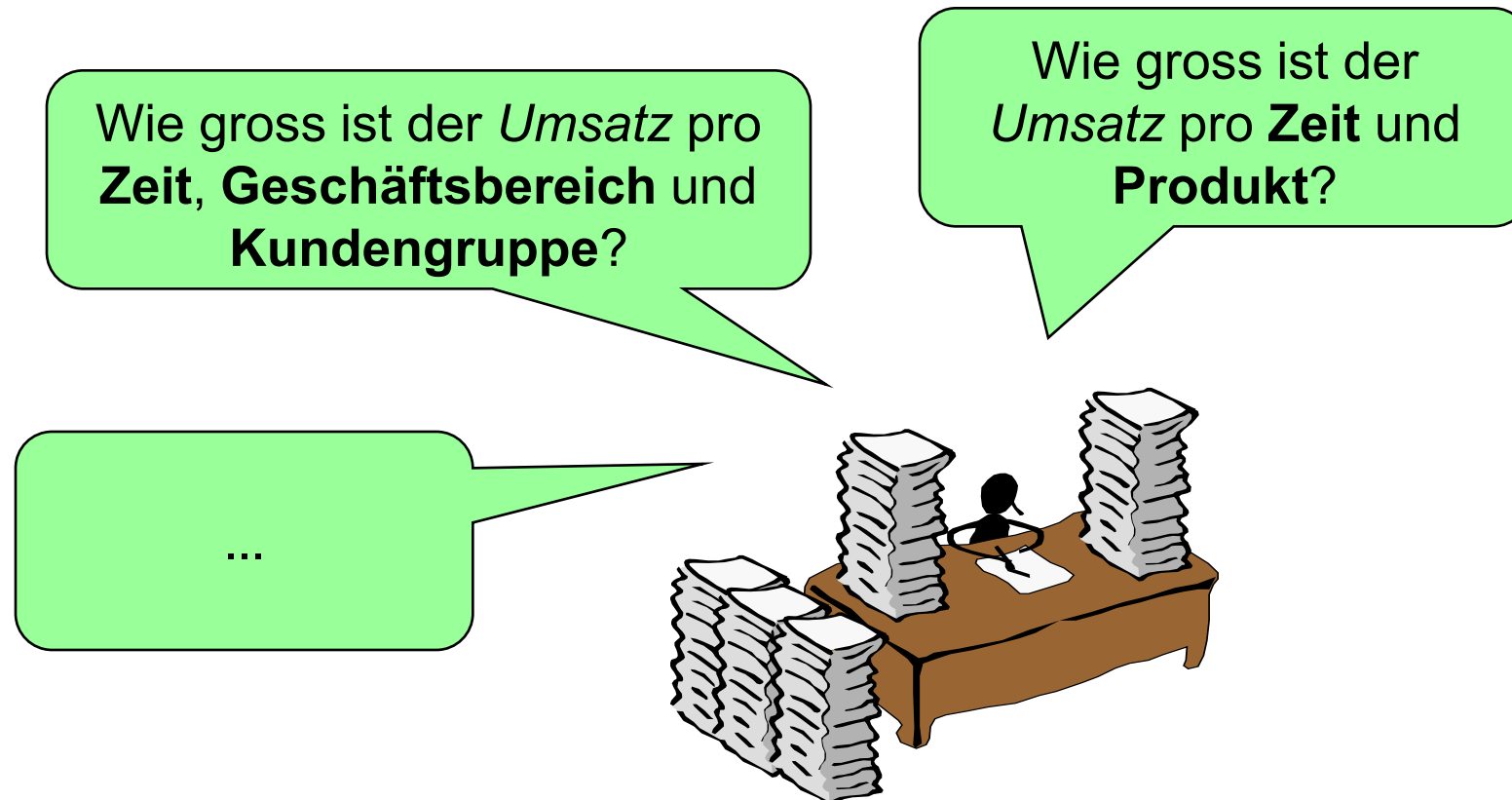
- Beispiele
 - Flugbuchungssystem
 - Bestellungen in einem Handelsunternehmen
 - Web-Shops
- Charakterisierung
 - Hoher Parallelitätsgrad
 - Viele (Tausende pro Sekunde) kurze Transaktionen
 - Transaktionen bearbeiten nur ein kleines Datenvolumen
 - „Mission-critical“ für das Unternehmen
 - Hohe Verfügbarkeit muss gewährleistet sein
 - Normalisierte Relationen (möglichst wenig Update-Kosten)
 - Nur wenige Indexe (wegen Fortschreibungskosten)

Einführung: Online Analytical Processing (OLAP)

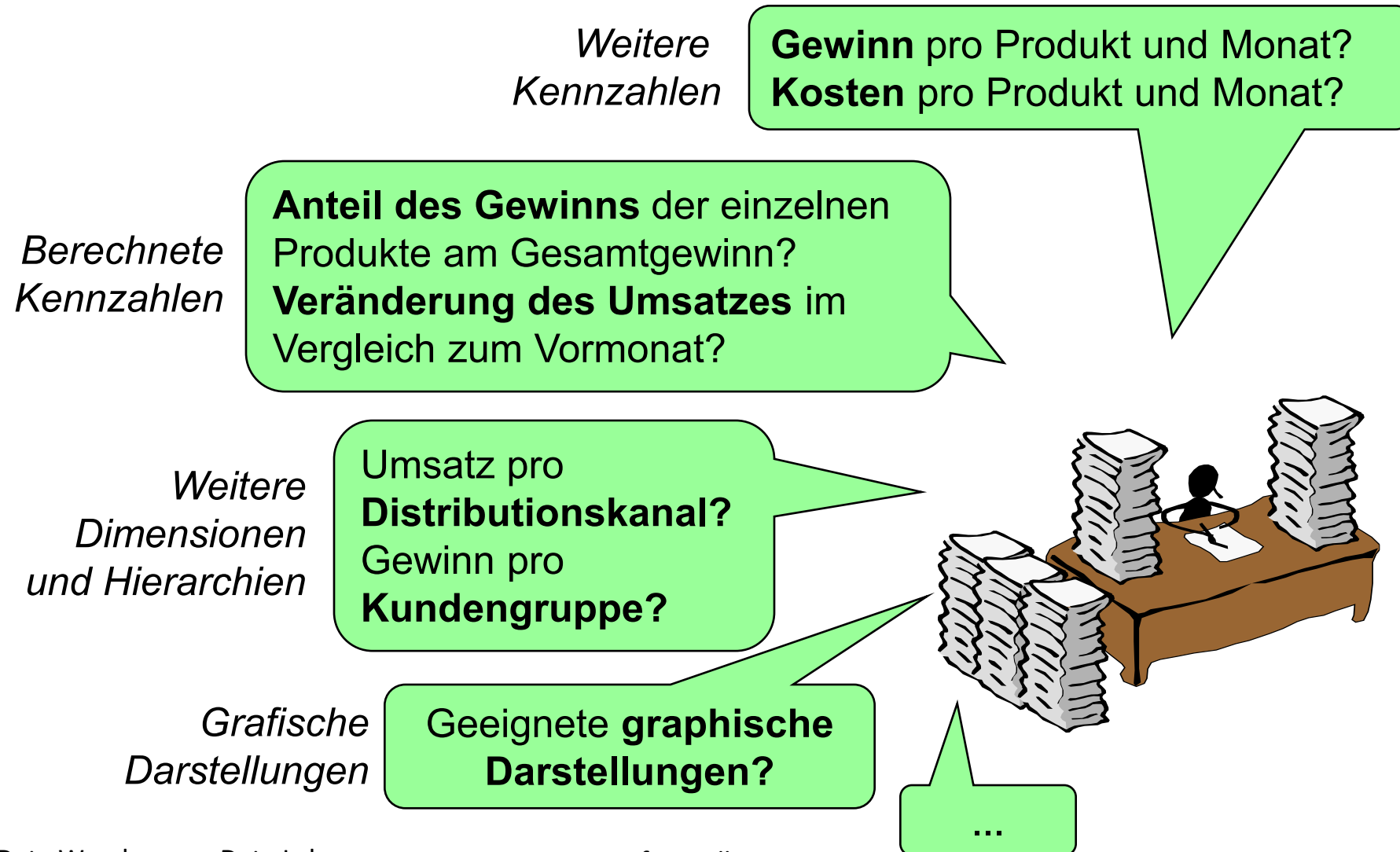
- Überbegriff für Technologien, Methoden und Tools zur Ad-hoc-Analyse multidimensionaler Informationen
- Die Daten werden nach unterschiedlichsten (ad-hoc) Kriterien ausgewertet
 - Daten werden nur ausgewertet und nicht verändert
 - Daten müssen nicht redundanzfrei vorliegen, es kann auf einer nicht ganz aktuellen Kopie gearbeitet werden

Einführung: Wofür wird OLAP benötigt? (1)

- OLAP erleichtert die Analyse von Kennzahlen unter verschiedenen Gesichtspunkten (Dimensionen)



Einführung: Wofür wird OLAP benötigt? (2)



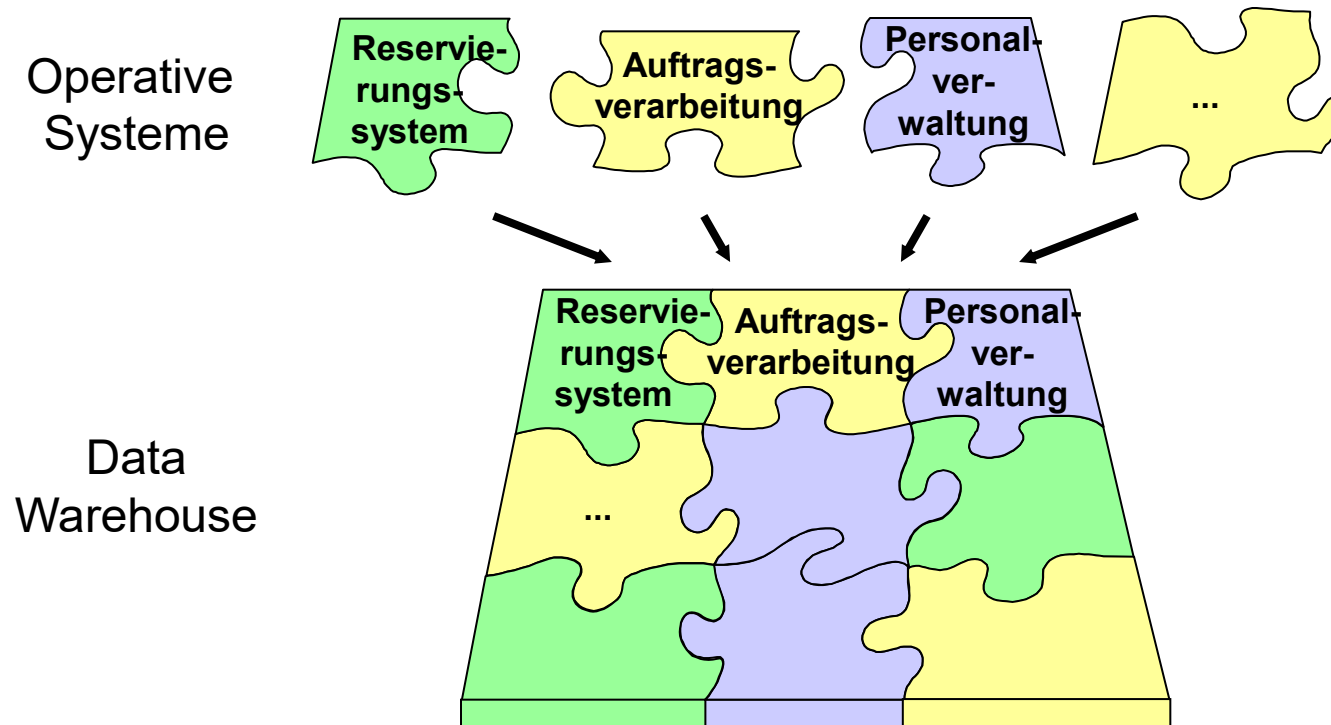
Einführung: Unterschiede OLTP-OLAP-Systeme

Transaktionsorientierte Systeme <i>Operative Systeme</i>	Auswertungsorientierte Systeme
OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Häufige, einfache Anfragen	Weniger häufige, komplexe Anfragen
Kleine Datenmengen je Anfrage	Grosse Datenmengen je Anfrage
Operieren hauptsächlich auf aktuellen Daten	Operieren auf aktuellen und historischen Daten
Schneller Update wichtig	Schnelle Kalkulation wichtig
→ Datenbanksystem kann nicht gleichzeitig für OLTP- und für OLAP-Anwendungen optimiert werden	
Paralleles Ausführung von OLAP-Anfragen auf operationalen Datenbeständen könnte Leistungsfähigkeit der OLTP-Anwendungen beeinträchtigen	

DATA WAREHOUSES (DW)

Einführung: Was ist ein Data Warehouse?

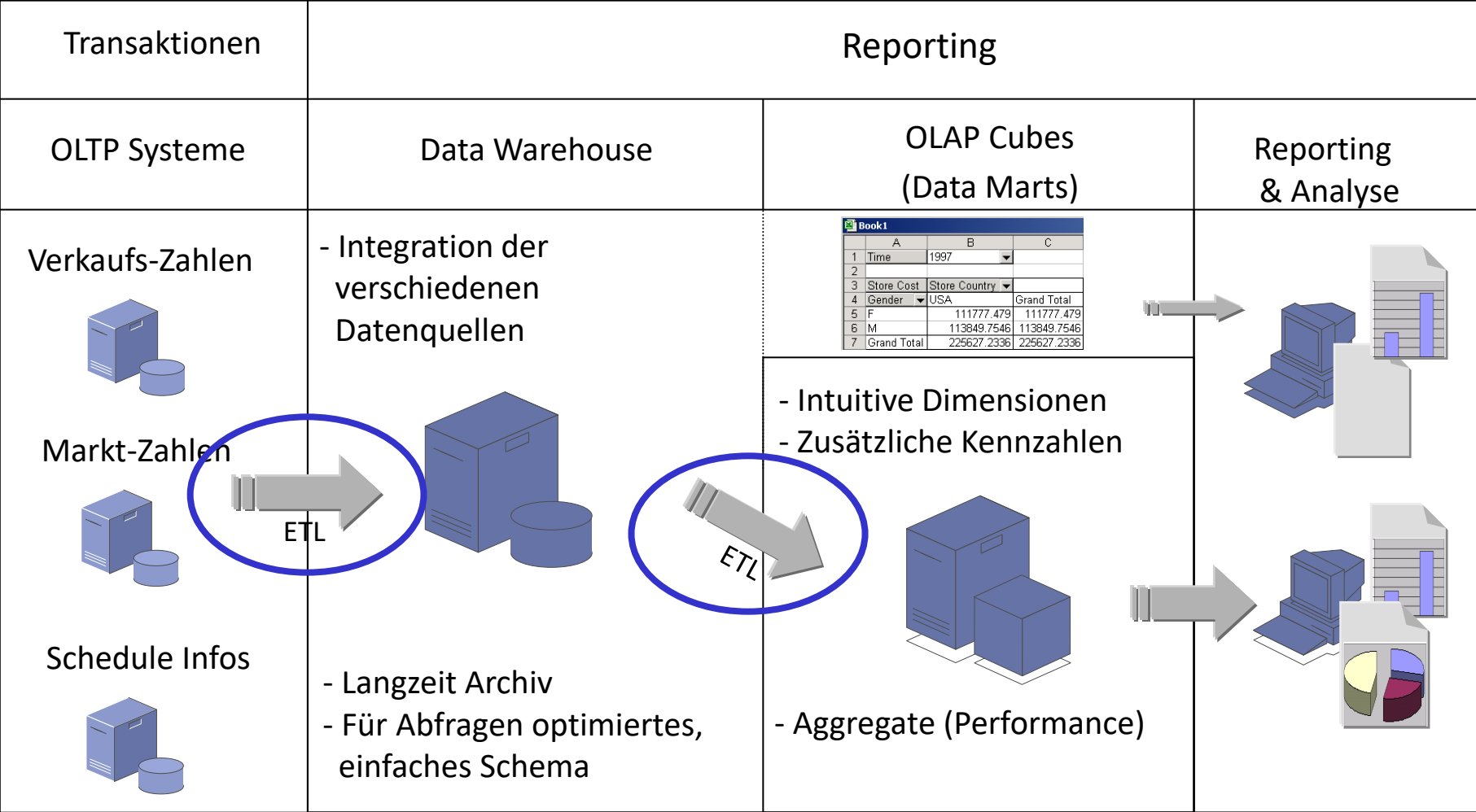
Ein Data Warehouse ist ein Datenbanksystem (für OLAP-Anwendungen), in dem die entscheidungsrelevanten Daten eines Unternehmens in konsolidierter Form gesammelt werden, um sie für (zeitbasierte) Auswertungen zugänglich zu machen.



Einführung: Was ist ein Data Warehouse? ff.

- Data Warehouse (DW): "... houses a standardized, consistent, clean and integrated form of data sourced from various operational systems in use in the organization, structured in a way to specifically address the reporting and analytic requirements". Data warehousing is a broader concept than Business intelligence.
- Business Intelligence (BI): "Skills, technologies, applications and practices used to help a business acquire a better understanding of its commercial context"
- BI Tools: Datenvisualisierung, u.a. Apache Superset, Tableau

OLAP Systemüberblick



- Extract-Transform-Load (ETL bzw. ELT)
 - Aufbereiten der Daten aus den versch. heterogenen Quellen
 - Laden der Daten ins DW, Aktualisieren der Daten im DW
- Data Warehouse (DW) (relational, Star, Snowflake)
- Data Mart: wohldefinierte Teilmenge des DW, aufbereitet + vorberechnete Aggregationen (relational)
- Auswertung/Darstellung der Daten im Data Warehouse oder Data Mart als Report, graphisch oder online mit Ad-Hoc-Queries (SQL oder MDX)
- (Data Lakes etc. im Ausblick)

Einführungsbeispiel Weinhandlung

- Eine Weinhandlung möchte ihre Verkaufsinformationen in einem Data Warehouse für Auswertungen ablegen
- Auswertungen auf dem Data Warehouse mit SQL
- (Auswertungen auf einem Data Cube (Data Mart))

Einführungsbeispiel: Wein-Shop Abfragen (1)

- Umsatz pro Zeit und Produkt

Umsatz							
	Jan	Feb	Mrz	Q1	Apr	...	2000
Merlot	33	55	56	144	18	...	760
Cabernet-S.	72	136	117	325	74	...	1338
Shiraz	85	128	99	312	92	...	1662
Rotweine	190	319	272	781	184	...	3760
Chardonnay	55	69	99	223	84	...	1051
Zinfandel	22	17	47	86	39	...	493
Weissweine	77	86	146	309	123	...	1544
Alle Produkte	267	405	418	1090	307	...	5304

Einführungsbeispiel: Wein-Shop Abfragen (2)

- Umsatz pro Zeit, Produkt und Region

Umsatz, Alle Regionen							
Umsatz, Oesterreich							
Umsatz, Deutschland							
Umsatz, Schweiz							
	Jan	Feb	Mrz	Q1	Apr	...	2000
Merlot	19	25	30	74	11	...	418
Cabernet-S.	48	71	54	173	44	...	702
Shiraz	40	82	35	157	39	...	955
Rotweine	107	178	119	404	94	...	2075
Chardonnay	25	34	22	81	33	...	356
Zinfandel	12	9	32	53	19	...	211
Weissweine	37	43	54	134	52	...	567
Alle Produkte	144	221	173	538	146	...	2642

Data-Warehouse Strukturen

- Star- und Snowflake- Schema
- Zeit-Dimension

Abfragen im relationalen Data Warehouse

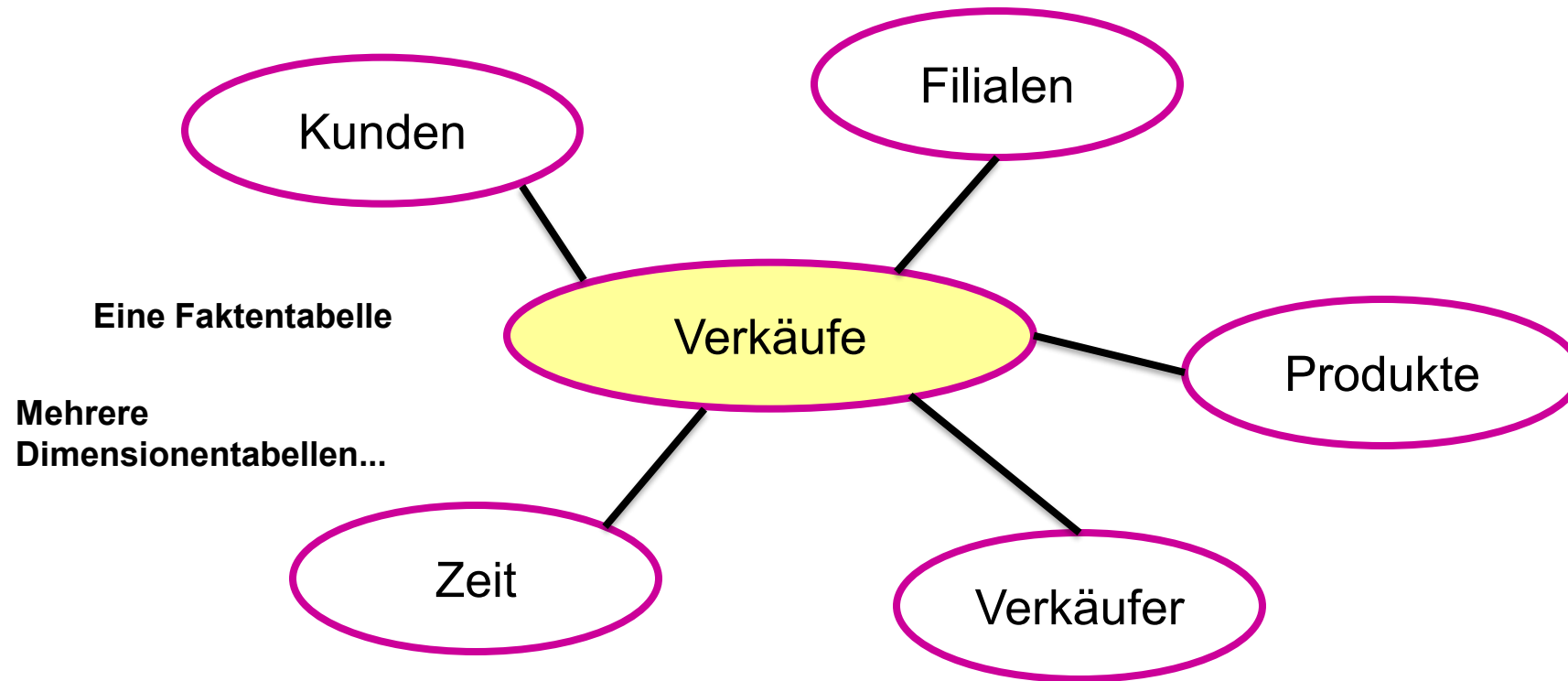
- GROUP BY
- ROLLUP, CUBE, GROUPING SETS
- (Analytische Funktionen)

RELATIONALE DATA WAREHOUSES

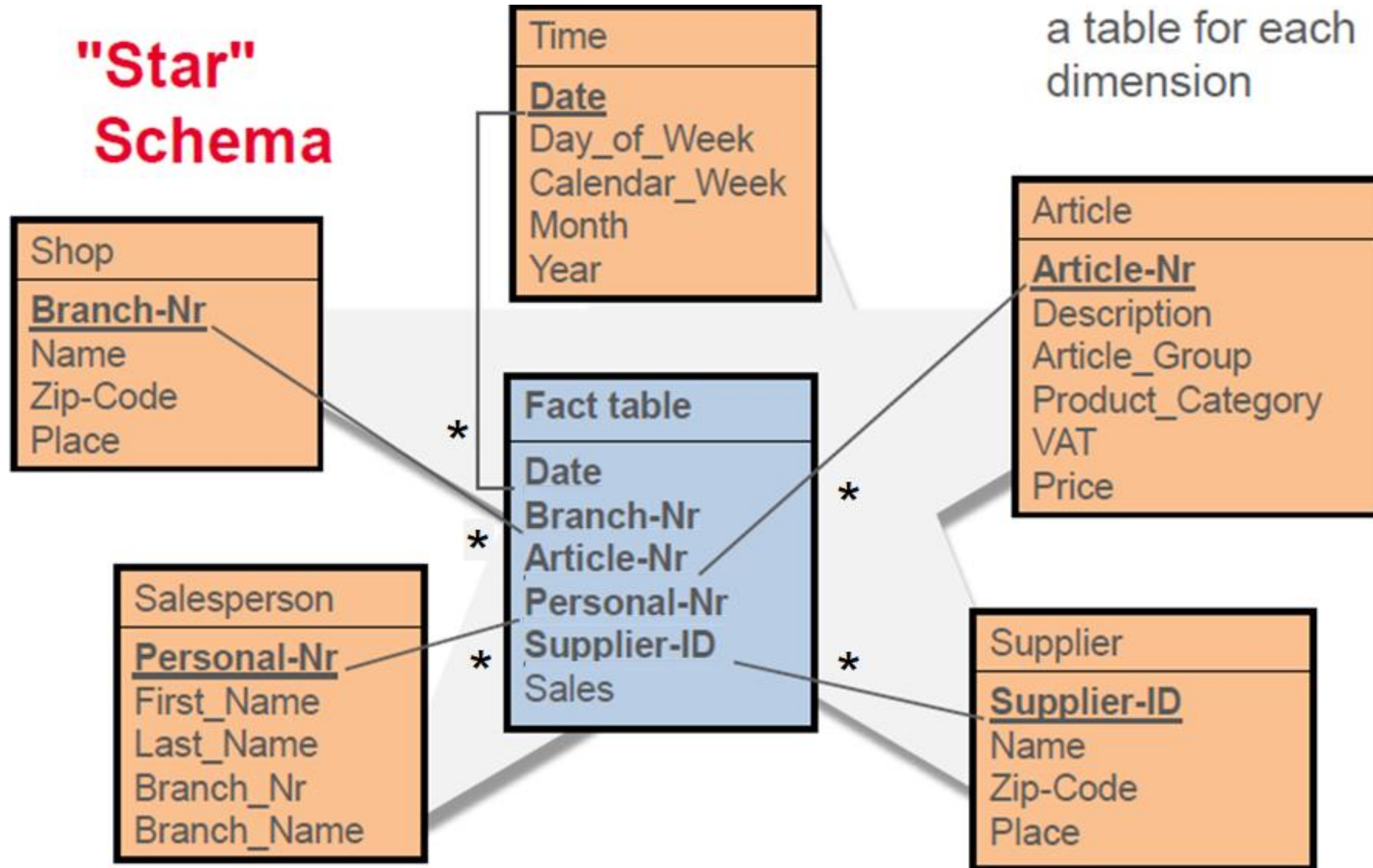
Stern-Schema bei DW-Anwendungen

- Faktentabelle
 - Bsp: Verkaufszahlen, Flugreservationen, Lagerbewegungen
 - normalisiert und typischerweise sehr gross bis mehrere Mio Tuples)
 - Bewegungsdaten (mit weiteren Attributen, die Referenzen/FKs auf Dimensionstabellen sind)
- Mehrere Dimensionstabellen
 - Zeit, Filialen, Kunden, Produkt
 - Oft nicht normalisiert und klein bis mittelgross (~ 1'000)
 - Aenderungshäufigkeit sehr klein
 - Beschreiben die Fakten in der Faktentabelle
- Abfragen: in der Regel 1-stufige Joins zwischen Fakten und verknüpften Dimensionen

Das Stern-Schema: Verkäufe



Beispiel Star-Schema

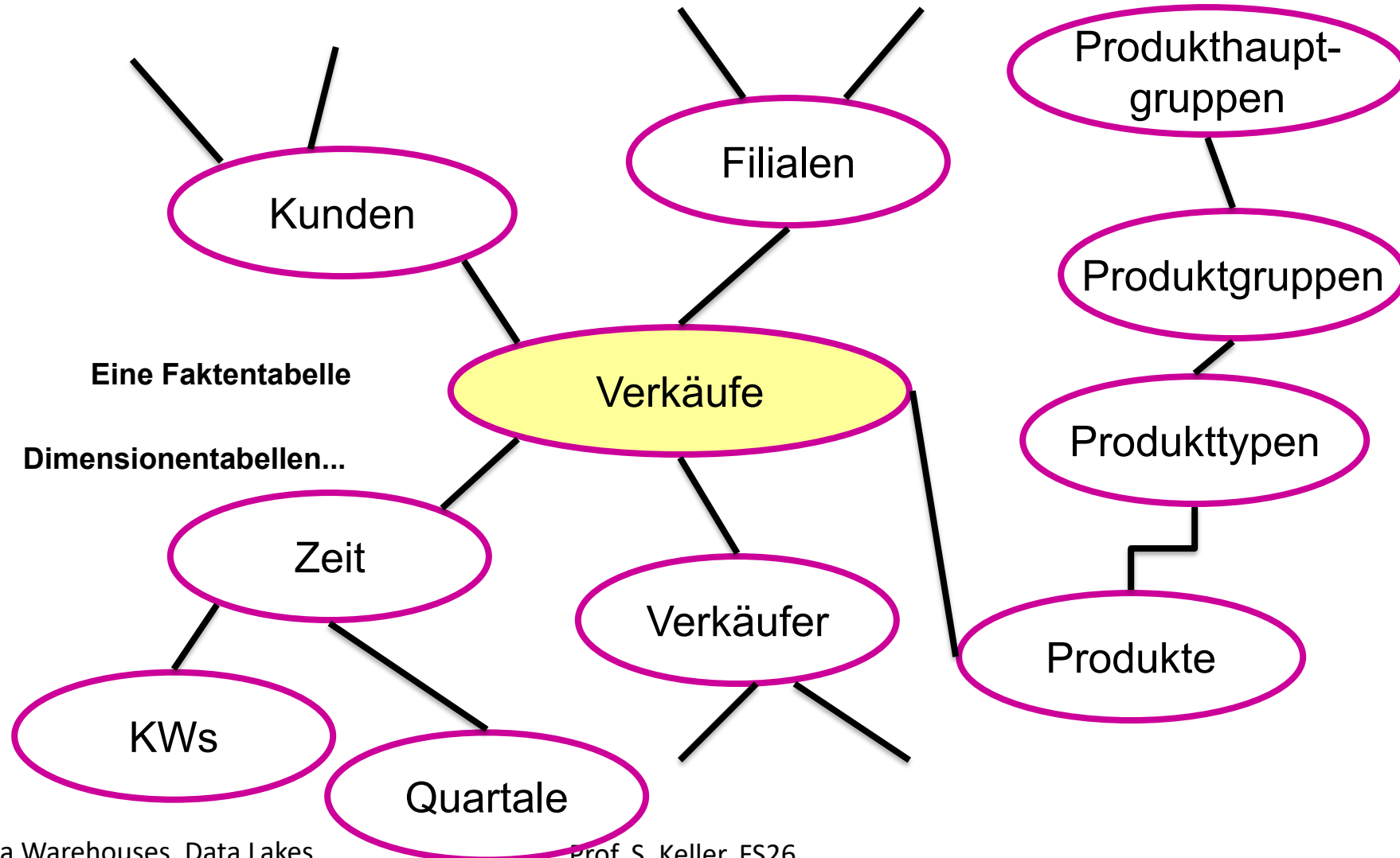


Snowflake-Schema

- Normalisierung der Dimensionen führt zu Snowflake-Schema
- pro Hierarchielevel in der Regel eine Tabelle
- Im Extremfall sind alle Dimensionen in 3.NF
- Mischformen Snowflake-Star möglich

Das Stern-Schema: Verkäufe

Normalisierung der Dimensionen führt zum Schneeflocken-Schema!



- "Data Warehouse Design" (englisch) – auch "Dimensional Modeling" genannt
- Faktentabellen enthalten nur numerische Werte und Fremdschlüssel auf Dimensionstabellen
- Faktentabellen sollte so granular wie möglich aufgebaut sein. Je granularer sie definiert sind, um so mehr Aggregate lassen sich zu einem späteren Zeitpunkt daraus bilden.
- Dimensionstabellen haben in der Regel einen Surrogat-Schlüssel
- Die Granularität der Zeit-Dimension kann nicht kleiner sein, als die Auflösung der Zeit im operativen System.
- „Conformed Dimensions“ haben (z.B. Zeit, Geographie, Kunde, Produkt) für alle möglichen Fact-Tables eine einheitliche Bedeutung und müssen daher besonders sorgfältig konzipiert werden. Ueber solche gemeinsamen Dimensionen sind Abfragen über mehrere Fakten-Tabellen möglich.
- Hierarchien in Dimensionen sind möglichst als funktionale Abhängigkeiten zwischen den Attributen einer flachen Dimensionstabelle (Star-Schema) oder als Fremdschlüsselbeziehungen zwischen Dimensionstabellen (Snowflake-Schema) zu konzipieren.

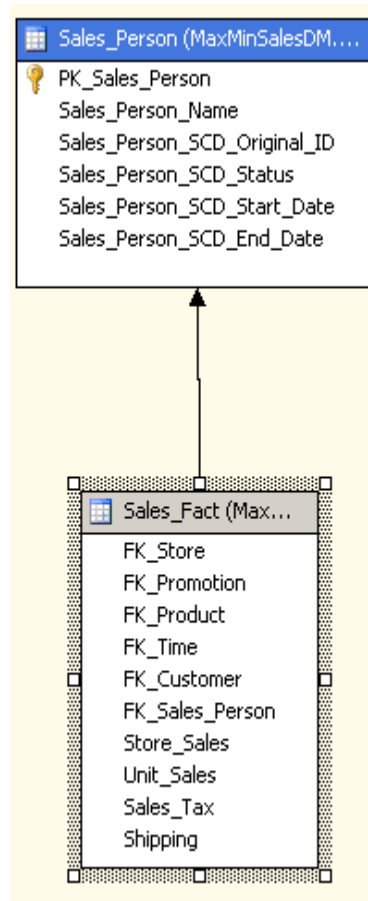
- Transaction Fact Table
 - Jeder Eintrag entspricht einem **Ereignis, das sich zu einem bestimmten Zeitpunkt** ereignete
 - Beispiel: Verkaufsfakten
- Periodic Snapshot Fact Tables
 - Jeder Eintrag enthält eine **kumulative Grösse einer Kennzahl**, gemessen in regelmässigen Intervallen
 - Beispiel: Lagerbestand: Der Lagerbestand der einzelnen Produkte wird z.B. jeden Abend erfasst
- Accumulating Snapshot Fact Tables
 - Jeder Eintrag beschreibt eine **Business-Transaktion, die aus mehreren Schritten** zu unterschiedlichen Zeitpunkten besteht
 - Beispiel: Bestellausführung (Eingang der Bestellung, Auslieferung der bestellten Produkte, Rechnungsausgang, Bestellungseingang)
 - Kann auch in Kombination mit Snapshot Fact Tables auftreten:
Beispiel: Monatlicher Snapshot wird aus den täglichen Transaktionen hergeleitet.

Data Warehouse Design: Spezielle Dimensionen

- Degenerate Dimension
 - Identifizieren eine umfassende Transaktion
 - Bsp: Order Header zu einer Order Line Item
- Role Playing Dimension
 - Eine Dimension wird von der Fact-Table mehrfach in unterschiedlichen Bedeutungen referenziert
 - Bsp: Zeit, Bestellzeit, Lieferzeit
- Junk Dimensions
 - Korrelierte Indikationen und Flags

- Slowly Changing Dimensions (SCD): "Veränderungen treten unerwartet, sporadisch und weitaus seltener auf als Messungen in Faktentabellen"
 - Bsp.: Verkäufer wechselt seine Region: Wie ist nun das Verhalten beim Aufsummieren der Verkaufsdaten? Oder: Preis wurde falsch eingetragen und muss korrigiert werden.
- SCD Typ 1: Keine Historisierung (Overwrite)
 - Nachteil: Auch Verkäufe im alten Gebiet zählen nun "plötzlich" zum neuen Gebiet
- SCD Typ 2: Historisierung mittels neuem Zeilen-Eintrag und Attribute PK_BK valid_from und valid_to (der "gängigste")
 - Alte Zuordnung bleibt erhalten (inkl. Zeitdauer der Änderung) zum "Preis", dass PK erweitert werden muss
 - Auswertungen können die History berücksichtigen, jedoch limitiert
- SCD Typ 3 bis 6:
 - Weitere Varianten der Historisierung durch zusätzliche Zeit-Attribute oder Tabellen mit verschiedenen Vorteilen (History), und Nachteilen (Komplexität)

DW Design: Bsp: SCD Typ 2



PK_Sales_Person: Surrogat-Key

Wichtig: Surrogat erforderlich, da für dieselbe Person mehrerer Einträge existieren können

Sales_Person_SCD_Original_ID: Businesskey (fachlicher Schlüssel) identifiziert Entität (hier Sales_Person) eindeutig

Start_Date ... Ende_Date: Gültigkeitsbereich des Eintrags

Status: Active oder Inactive (redundant zum Gültigkeitsbereich)

	PK_Sales_Person	Sales_Person_Name	Sales_Person_SCD_Original_ID	Sales_Person_SCD_Status	Sales_Person_SCD_Start_Date	Sales_Person_SCD_End_Date
1	1	Anne	100	Inactive	NULL	2006-03-30 23:59:59.000
2	2	Franke	200	Active	NULL	9999-12-31 00:00:00.000
3	11	Anne	100	Active	2006-04-01 00:00:00.000	9999-12-31 00:00:00.000

Beispiel SCD Typ 2

Dimensionstabelle *Produkt* : Preis als SCD 2 Attribut

Produkt_ID	Product_ID_BK	ProduktName	Preis	Weinart_ID	scdValidFrom	scdValidTo
1	1	Chardonnay	25.50	2	2004-01-01	NULL
2	2	Zinfandel	18.90	2	2004-01-01	NULL
3	3	Cabernet-S.	32.00	1	2004-01-01	2007-06-06
4	4	Shiraz	21.00	1	2004-01-01	NULL
5	5	Merlot	16.00	2	2004-01-01	NULL
6	3	Cabernet-S.	33.00	1	2007-06-07	NULL

Bsp: Fakt-Tabelle (vereinfacht)

Date	Produkt_ID	Anzahl
2006-03-24	3	23
2006-04-27	1	34
2007-06-06	3	22
2007-06-07	6	34

Auswertung über Business-Key (BK)

Select SUM(Anzahl) From FaktTab
INNER JOIN DimProdukt ON ...
GROUP BY **Produkt_Id_BK**

Auswertung über Primary Key (PK historisierend)

Select SUM(Anzahl*Preis) From FaktTab
INNER JOIN DimProdukt ON ...
GROUP BY **Produkt_Id**

Bemerkung: Beim Aufbau der Fakttabelle muss das jeweils richtige Tupel der Dim-Tabelle via Business-Key und scdValidTo gesucht

Temporales Datenmanagement in DBMS

- MS SQL Server und Azure SQL Database verfügen über integrierte "system-versioned temporal tables", die SCD Typ 2 implementieren.
- Oracle und DB2 verfügen über eine integrierte Verwaltung zeitlicher Daten, SCD Typ 2 teilweise implementieren.
- PostgreSQL hat kein eingebautes temporales Datenmanagement. Es gibt aber Alternativen "von Hand".
 - Siehe z.B. dieser Blogbeitrag: "The Ultimate Guide to PostgreSQL Data Change Tracking", Feb 23 2024 by <https://exaspark.medium.com/the-ultimate-guide-to-postgresql-data-change-tracking-c3fa88779572>

	Audit Triggers	Notify/Listen	Application Tracking	CDC	Integrated CDC
Reliability and accuracy	✓	✗	✗	✓	✓
Minimal performance impact	✗	✓	✗	✓	✓
Initial implementation simplicity	✓	✓	✓	✗	✗
Scalability	✗	✗	✗	✓	✓
Data change contextualization	✓	✓	✓	✗	✓

- Charakterisierung der Faktentabelle
- FaktBestellung:
 - Granularität Tag, „Accumulating Snapshot Table“
- FaktBudget:
 - Granularität Halbjahr, „Transaction Fact Table“
- Charakterisierung der Dimensionen
 - Degenerierte Dimension (BestellItemNr) in der Tabelle FaktBestellung
 - DimZeit ist Role-Playing Dimension zu Faktbestellung (Role BestellZeit und Lieferzeit)
 - DimProdukt: Preis und ProduktName könnte historisiert werden mit SCD Typ 2 (Ausbau)
- (Mehr dazu im Anhang)

AGGREGATIONS-FUNKTIONEN IN SQL

Anfragen im Stern-Schema: Aggregation in SQL

- Aggregatfunktionen:
 - Mit COUNT(), SUM(), MIN(), MAX(), AVG(), ...
 - Aggregatfunktionen liefern einen einzelnen Wert
- Aggregation über mehrere Attribute
 - mit GROUP BY
- Komplexere Gruppierungen
 - mit ROLLUP, CUBE and GROUPING SETS-Operatoren

Anfragen im Stern-Schema: Beispiel Autoverkäufe

Modell	Jahr	Farbe	Anzahl
Opel	1990	rot	5
Opel	1990	weiß	87
Opel	1990	blau	62
Opel	1991	rot	54
Opel	1991	weiß	95
Opel	1991	blau	49
Opel	1992	rot	31
Opel	1992	weiß	54
Opel	1992	blau	71
Ford	1990	rot	64
Ford	1990	weiß	62
Ford	1990	blau	63
Ford	1991	rot	52
Ford	1991	weiß	9
Ford	1991	blau	55
Ford	1992	rot	27
Ford	1992	weiß	62
Ford	1992	blau	39

Anfragen im Stern-Schema: GROUP BY

```
SELECT modell, jahr, SUM(anzahl)  
FROM autoverkaeufe  
GROUP BY modell, jahr  
ORDER BY 1,2;
```

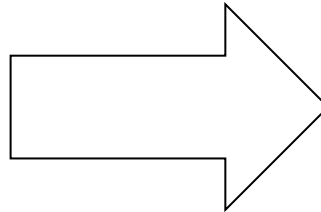
modell	jahr	sum
-----	----	---
Ford	1990	189
Ford	1991	116
Ford	1992	128
Opel	1990	154
Opel	1991	198
Opel	1992	156

- Die Tabelle wird gemäss GROUP BY gegliedert
- Jede Gruppe wird über eine Funktion (hier "SUM") aggregiert
- Das Resultat ist eine Tabelle mit aggregierten Werten

Anfragen im Stern-Schema: Beispiel GROUP BY

```
SELECT modell, jahr, SUM(anzahl) "anz"  
FROM autoverkaeufe  
GROUP BY modell, jahr
```

Modell	Jahr	Anzahl
Opel	1990	154
Opel	1991	198
Opel	1992	156
Ford	1990	189
Ford	1991	116
Ford	1992	128



Modell	Jahr	Farbe	Anzahl
Opel	1990	rot	5
Opel	1990	weiß	87
Opel	1990	blau	62
Opel	1991	rot	54
Opel	1991	weiß	95
Opel	1991	blau	49
Opel	1992	rot	31
Opel	1992	weiß	54
Opel	1992	blau	71
Ford	1990	rot	64
Ford	1990	weiß	62
Ford	1990	blau	63
Ford	1991	rot	52
Ford	1991	weiß	9
Ford	1991	blau	55
Ford	1992	rot	27
Ford	1992	weiß	62
Ford	1992	blau	39

GROUP BY ROLLUP

- Gleiche Anfrage in unterschiedlichen Detaillierungsgraden
- Verminderung des Detaillierungsgrades = Roll-Up
- (Gegenteil: Erhöhung des Detaillierungsgrades = Drill-Down)
- Beispiel Autoverkäufe:
 - Roll Up über zwei „Ebenen“ (Felder Modell und Jahr):
GROUP BY ROLLUP(modell, jahr)
 - Daten werden nach Modell, dann nach Jahr aggregiert (plus Total):
 - Zuerst die Verkaufszahlen für jedes Modell pro Jahr summiert (wie „normales“ GROUP BY),
 - dann werden alle Verkaufszahlen des Modells über alle Jahre summiert (mehrere Einzeiler)
 - und daraus die Verkaufszahlen aller Modelle über alle Jahre summiert (ein einziger Einzeiler)

Beispiel GROUP BY ROLLUP(...) rein mit GROUP BY

```
SELECT modell, jahr, sum(anzahl) AS "anz"
FROM autoverkaeufe
GROUP BY modell, jahr
ORDER BY modell, jahr;
-- Query und Daten wie bisher
```

modell	jahr	anz
-----	-----	-----
Ford	1990	189
Ford	1991	116
Ford	1992	128
Opel	1990	154
Opel	1991	198
Opel	1992	156

```
SELECT modell, jahr, sum(anzahl) "anz"
FROM autoverkaeufe
GROUP BY modell, jahr
UNION ALL
SELECT modell, null, sum(anzahl)
FROM autoverkaeufe
GROUP BY modell
UNION ALL
SELECT null, null, sum(anzahl)
FROM autoverkaeufe
ORDER BY modell, jahr;
```

modell	jahr	anz
-----	-----	-----
Ford	1990	189
Ford	1991	116
Ford	1992	128
Ford	(null)	433
Opel	1990	154
Opel	1991	198
Opel	1992	156
Opel	(null)	508
(null)	(null)	941

Probleme mit GROUP BY: ROLLUP-Operator

- Roll Up ist asymmetrisch: Verkäufe sind nach Jahr, aber nicht nach Farbe aggregiert.
- Resultierende Tabelle ist nicht relational, da man wegen der leeren Felder (NULL/ALL-Werte) keinen Schlüssel festlegen kann.
- Die Zahl der Spalten wächst mit der Zahl der aggregierten Attribute
- Der NULL/ALL-Wert repräsentiert die Menge, über die die Aggregation berechnet wird.
 - Autoverkäufe-Beispiel:
Ein NULL/ALL in der Spalte Farbe bedeutet, dass in der Anzahl dieser Zeile die Verkaufszahlen der roten, weissen und blauen Autos zusammengefasst sind.
- Eine Aggregation über n Dimensionen erfordert n Unions →
ROLLUP-Operator leistet dasselbe:

```
SELECT modell, jahr, SUM(anzahl) as "anz"  
FROM autoverkaeufe  
GROUP BY ROLLUP (modell, jahr);
```

- Beispiel:

```
SELECT modell, jahr, SUM(anzahl) "anz"  
FROM autoverkaeufe  
GROUP BY CUBE (modell, jahr, farbe);
```

- Berechnet Subtotale für alle möglichen Kombinationen der Attribute
- Der Cube-Operator erzeugt eine Tabelle, die sämtliche Aggregationen enthält
- Die Erzeugung der Tabelle erfordert die Generierung der Potenzmenge der zu aggregierenden Spalten:
 - Bei n Attributen gibt es $(2^n)-1$ Subtotal-Kombinationen, z.B.
GROUP BY CUBE(modell, jahr, farbe) => 3 Attribute: $(2^3) - 1 => 7$
=> modell, jahr, farbe, modell+jahr, modell+farbe, jahr+farbe, modell+jahr+farbe

Beispiel GROUP BY CUBE vs. GROUP BY ROLLUP

```
SELECT modell, jahr, SUM(anzahl) "anz"
FROM autoverkaeufe
GROUP BY CUBE (modell, jahr)
ORDER BY 1,2;
```

modell	jahr	anz
Ford	1990	189
Ford	1991	116
Ford	1992	128
Ford	(null)	433
Opel	1990	154
Opel	1991	198
Opel	1992	156
Opel	(null)	508
(null)	1990	343
(null)	1991	314
(null)	1992	284
(null)	(null)	941

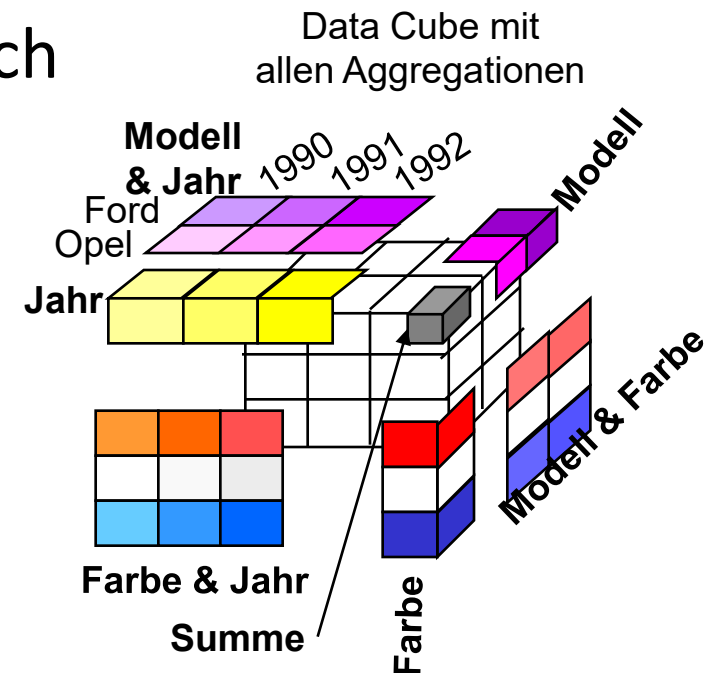
```
SELECT modell, jahr, sum(anzahl) "anz"
FROM autoverkaeufe
GROUP BY ROLLUP (modell, jahr)
ORDER BY 1,2;
```

modell	jahr	anz
Ford	1990	189
Ford	1991	116
Ford	1992	128
Ford	(null)	433
Opel	1990	154
Opel	1991	198
Opel	1992	156
Opel	(null)	508
(null)	(null)	941

Der CUBE-Operator

- N-dimensionale Generalisierung der bisher genannten Konzepte
- Der 0D Data Cube ist ein Punkt
- Der 1D Data Cube ist eine Linie mit einem Punkt
- Der 2D Data Cube ist eine Kreuztabelle
- Der 3D Data Cube ist ein Würfel mit drei sich überschneidenden Kreuztabellen

GROUP BY (mit Gesamtsumme)		
Aggregation		
Summe		
	rot	
	weiss	
	blau	
Summe		
	rot	
	weiss	
	blau	
Modell		
	Opel	
	Ford	
	Farbe	

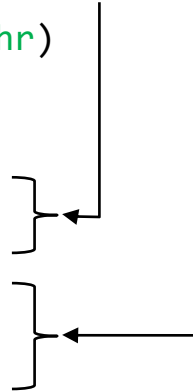


Der (universelle) GROUPING SETS-Operator

Gib ausdrücklich an, welche Zwischensummen benötigt werden

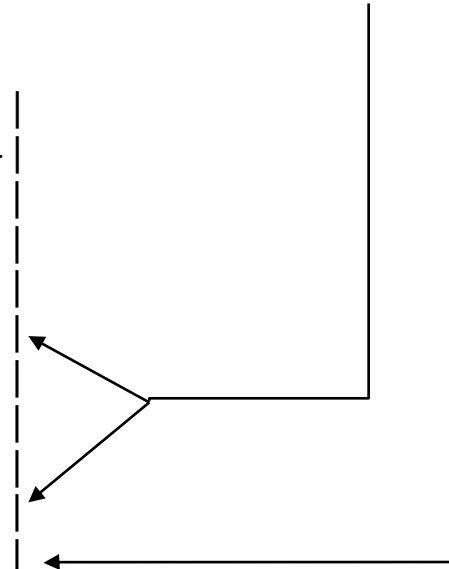
```
SELECT modell, jahr, SUM(anzahl) "anz"
FROM autoverkaeufe
GROUP BY GROUPING SETS (modell, jahr)
ORDER BY 1,2;
```

modell	jahr	anz
-----	-----	-----
Ford	(null)	433
Opel	(null)	508
(null)	1990	343
(null)	1991	314
(null)	1992	284



```
SELECT modell, jahr, sum(anzahl) "anz"
FROM autoverkaeufe
GROUP BY GROUPING SETS ((modell, jahr),modell,())
ORDER BY 1,2;
```

modell	jahr	anz
-----	-----	-----
Ford	1990	189
Ford	1991	116
Ford	1992	128
Ford	(null)	433
Opel	1990	154
Opel	1991	198
Opel	1992	156
Opel	(null)	508
(null)	(null)	941



Das ist identisch mit dem ROLLUP-Beispiel vorhin:

```
SELECT modell, jahr, sum(anzahl) "anz"
FROM autoverkaeufe
GROUP BY ROLLUP (modell, jahr)
ORDER BY 1,2;
```

ROLLUP, CUBE, GROUPING SETS: Zusammenfassung

- GROUP BY ...
 - ROLLUP: Erstellt Gruppensummen von rechts nach links und eine Gesamtsumme
 - CUBE: Wie ROLLUP mit allen möglichen Kombinationen
 - GROUPING SETS: Gibt explizit an, welche Zwischensummen benötigt werden
- Generieren ähnliche Resultate, sind nicht „orthogonal“
- Sie sind v.a. zur effizienten Berechnung gedacht

PIVOT-TABELLEN

- "Pivot" engl. "Drehen", Pivotieren = Zeilen in Spalten umwandeln
- Tabellen-Werte einer Spalte sollen in Spalten mit aggregierten Werten umgewandelt ("transponiert") werden:
 - Zum Zusammenfassen, Analysieren und Präsentieren grosser Datenmengen
 - Typischerweise in Statistik und Marktforschung verwendet
 - Aggregation = SUM, COUNT etc.
- Pivot- und Kreuz- Tabellen sind praktisch synonym:
 - Pivot-Tabelle: Flexibler und interaktiver als eine Kreuztabelle
 - Kreuz-Tabellen: Starrer in ihrer Struktur als eine Pivot-Tabelle
 - Die UNPIVOT-Anweisung ist die Umkehrung der PIVOT-Anweisung

Pivot: Beispiel Autoverkäufe pro Jahr

- Gegeben: Die Daten / Tabelle Autoverkäufe (Folie 36)
- Gesucht: Jahre 1990, 1991, 1992 als eigene Spalten mit den Summen

```
SELECT modell, jahr, SUM(anzahl) AS "anz"
FROM autoverkaeufe
GROUP BY modell, jahr
ORDER BY 1,2;
```

modell	jahr	anz
Ford	1990	189
Ford	1991	116
Ford	1992	128
Opel	1990	154
Opel	1991	198
Opel	1992	156

modell	1990	1991	1992
Ford	189	116	128
Opel	154	198	156

Pivot: Mit FILTER-Klausel und mit PostgreSQL crosstab

Mit FILTER-Klausel (SQL:2023)

- Wird mit mehr Attributen und/oder anderen Datentypen schnell unhandlich
- Unten nochmals das Resultat

```
SELECT
  modell,
  SUM(anzahl) FILTER (WHERE jahr=1990) AS "1990",
  SUM(anzahl) FILTER (WHERE jahr=1991) AS "1991",
  SUM(anzahl) FILTER (WHERE jahr=1992) AS "1992"
FROM autoverkaeufe
GROUP BY modell
ORDER BY 1,2;
```

-- Ausgabe:

modell	1990	1991	1992
Ford	189	116	128
Opel	154	198	156

Mit der Table-Function crosstab

- Zwei-Parameter-Fn.-Version:
 - oben mit expliziten Spaltennamen
SELECT unnest('{1990,1991...}',
 - unten etwas dynamischer:
SELECT DISTINCT jahr...)

```
SELECT *
FROM crosstab(
  $$ SELECT modell, jahr, SUM(anzahl) FROM autoverkaeufe
      GROUP BY modell, jahr ORDER BY 1,2 $$,
  $$ SELECT unnest('{1990,1991,1992}'::text[]) $$
) AS tmp(modell text, "1990" int, "1991" int, "1992" int);
```

```
SELECT *
FROM crosstab(
  $$ SELECT modell, jahr, SUM(anzahl) FROM autoverkaeufe
      GROUP BY modell, jahr ORDER BY 1,2 $$,
  $$ SELECT DISTINCT jahr FROM autoverkaeufe ORDER BY 1 $$
) AS tmp(modell text, "1990" int, "1991" int, "1992" int);
```

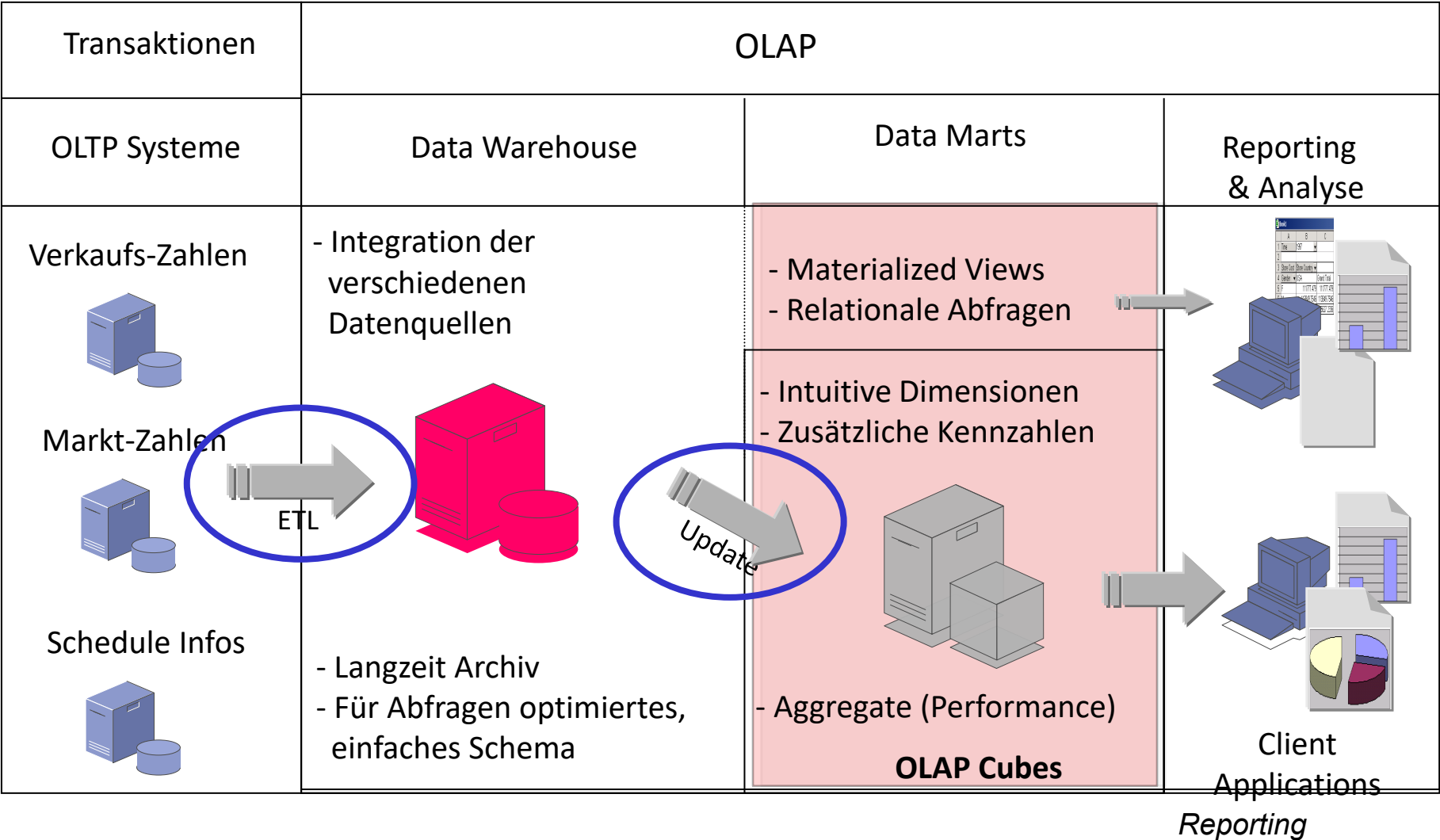
- Auftrennen von Werten innerhalb einer Spalte in eigene Spalten.
- Die Werte in diesen neuen Spalten werden mithilfe einer Aggregatfunktion für die Teilmenge der Zeilen berechnet, die mit jedem eindeutigen Wert übereinstimmen.
- Statisches PIVOT vs. Dynamisches PIVOT
 - "Pain point": Beim statischen PIVOT muss man für alle PIVOT-Werte (also z.B. Jahre 2020, 2021, 2022) Spaltenname und -typ "von Hand" einzeln aufzählen.
 - Lösung: Dynamisches PIVOT. Multisets oder Dokumenttypen (XML, JSON) verwenden.
- SQL Standard: Eine Alternative zu FILTER ist CASE
 - Wie vorheriges FILTER-Beispiel jedoch mit COALESCE(SUM(anzahl) FILTER (WHERE jahr=1990), 0) AS "1990,..."

Pivot: Implementationen

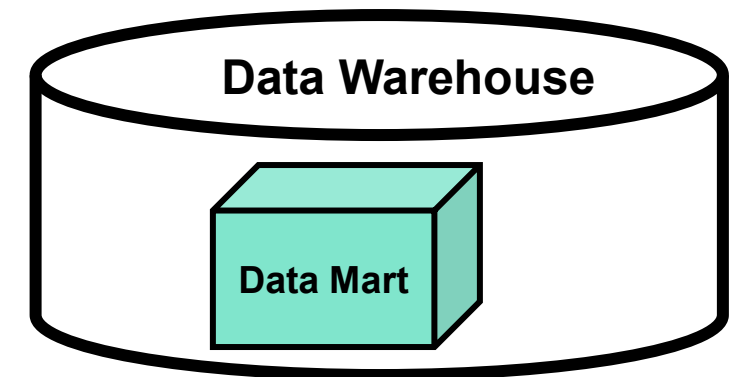
- Excel und Tableau:
 - Pivotieren durch "Ziehen" von Feldern mit der Maus
- Oracle, MS SQL Server, SQLite / DuckDB:
 - PIVOT und UNPIVOT als proprietäre Alternativen
- PostgreSQL: Funktion "crosstab" der Extension "tablefunc":
 - Installation: CREATE EXTENSION tablefunc;
 - Fn. crosstab erzeugt eine "Pivot-Tabelle" mit Zeilennamen und N Wertespalten, wobei N durch den in der Query angegebenen Zeilentyp bestimmt wird.
 - Doku.: <https://www.postgresql.org/docs/current/tablefunc.html>
 - Tipp: <https://postgresql.verite.pro/blog/2018/06/19/crosstab-pivot.html>
- CASE ist in praktisch allen DBMS implementiert
- FILTER: PostgreSQL, SQL Server, SQLite/DuckDB, Oracle mit "KEEP"

DATA MARTS

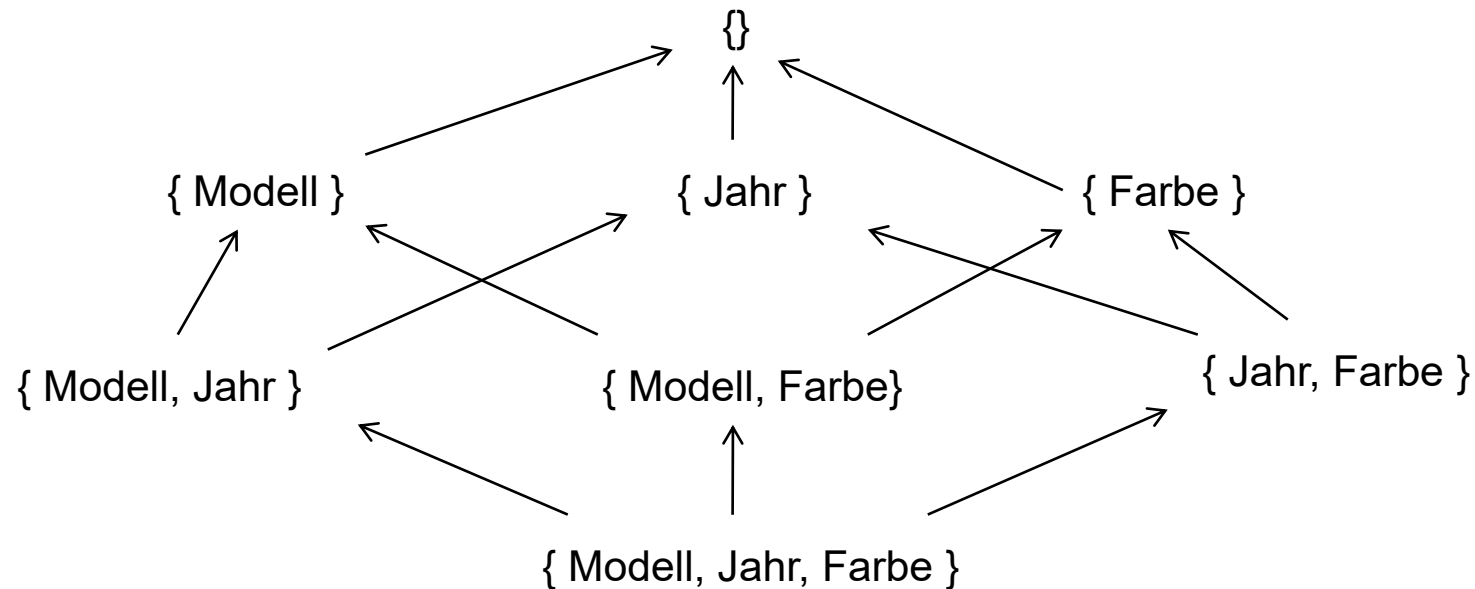
Einführung - OLAP Systemüberblick



- Teilmenge eines grösseren DW, das für eine bestimmte Geschäftseinheit innerhalb eines Unternehmens konzipiert ist, z.B. Vertrieb, Marketing, Finanzen
- Data Marts sind kleiner und in der Regel flexibler als DW
- Data Marts können strukturierte oder unstrukturierte Daten enthalten
- Ermöglichen schnellere Zugriffe und optimierte Abfragen durch Vorberechnung und Speicherung
 - relational z.B. als materialisierte View
 - oder in speziellen Cube-Datenstrukturen



Die Materialisierungs-Hierarchie im Data Mart



- Teilaggregate T sind für eine Aggregation A wiederverwendbar, wenn es einen gerichteten Pfad von T nach A gibt
- Also $T \rightarrow \dots \rightarrow A$
- Man nennt diese Materialisierungshierarchie auch einen Verband (Engl. *Lattice*)

Bewertung: Relationale Abfragen auf Data Mart

- Optimierung aufwändig
- Viele Abfragen (Slice, Dice, Vergleich mit einer anderen Zeitperiode, Navigation in den Hierarchien etc.) sind komplex, werden zudem nicht von allen Produkten unterstützt
- Metadaten über Dimensionen, Fakten müssen explizit verwaltet werden (einzelne Hersteller bieten relationale Unterstützung z.B. für die Beschreibung der Dimensionen an)

➔ Verbesserung: Data Cube! Würfel-artige Datenstruktur mit eigener Abfragesprache (z.B. MDX)

Ausblick und Abschluss

Data Warehouse, Data Lake, Data Mesh, Data Space ...

- Data Warehouse (DW) und Data Mart:
 - Zentrales Repository für strukturierte Daten inkl. deren Aggregationen
 - Entwickelt, um Business Intelligence und Entscheidungsfindungs-Aktivitäten zu unterstützen
 - Bietet eine einzige "Quelle der Wahrheit" für Unternehmensdaten
 - Data Mart: Teilmenge eines grösseren DW (siehe vorheriges Kapitel)
- Data Lake:
 - Speichersystem für strukturierte und unstrukturierte Daten
 - Speichert grosse Mengen an Rohdaten in ihrem ursprünglichen Format, bis sie benötigt werden
 - Entwickelt zur Unterstützung von Big Data-Verarbeitung, maschinellem Lernen und Analyse
- Data Mesh:
 - Dezentraler Ansatz zur Verwaltung von Daten
 - Daten werden wie ein Produkt behandelt, das von einzelnen Teams verwaltet wird
 - Betont die Bedeutung der Datenautonomie, des bereichsbezogenen Designs und der Demokratisierung von Daten
 - Teams sind für die Qualität, Sicherheit und Verwaltung ihrer eigenen Datenprodukte verantwortlich
 - Datenprodukte werden anderen Teams über eine gemeinsame Datenplattform zur Verfügung gestellt
- Data Space:
 - Definition: Föderierte Datenökosysteme, in denen verschiedene Akteure Daten teilen
 - Hintergrund: Initiativen wie GAIA-X in Europa; Fokus auf Datensouveränität, Interoperabilität, Sicherheit
 - Föderierte Architektur: Keine zentrale Datenablage, stattdessen Datenaustausch über standardisierte Schnittstellen
 - Herausforderungen: Rechtliche Aspekte, Vertrauensmodelle, Einigung auf Metadatenstandards

Data Warehouse, Data Lake, Data Mesh, Data Space ...

Merkmal	Data Warehouse	Data Lake	Data Mesh	Data Space
Zweck/Fokus	Zentralisiertes System für konsolidierte Analysen	Rohdaten-Speicherung für flexible Analysen	Dezentralisierte Datenverwaltung in Domänen	Föderierter Datenaustausch zwischen Organisationen
Strukturierung	Vordefinierte Schemata (Schema-on-Write)	Schema-on-Read, keine Vorab-Transformation	Daten als Produkt jeder Domäne	Föderierte Architektur, keine zentrale Kopie
Governance	Zentrale Kontrolle durch BI/IT-Abteilung	Kaum Vorgaben, Gefahr eines ‚Data Swamp‘	Verantwortung bei den Fach-Domänen + zentrale Standards	Gemeinsame Richtlinien, Datensouveränität im Vordergrund
Vorteile	Verlässliche Datenqualität, Reporting	Grosse Flexibilität, unstrukturierte Daten möglich	Hohe Agilität, Fachkompetenz in Datenhaltung	Souveräner Austausch über Unternehmensgrenzen hinweg
Nachteile	Aufwendige ETL-Prozesse, weniger flexibel	Fehlende Governance, mögliche Datenüberflutung, ETL delegiert	Höherer Abstimmungsaufwand, klare Rollen nötig	Rechtsfragen, Koordination übergreifend, fehlende Standards

Die Lernziele waren

- Sie kennen die Grundlagen und Anwendungen von Data Warehouses und OLAP
- Sie können einfache relationale Data Warehouses entwerfen: Spezielle Dimensionen / Slowly Changing Dimensions (SCD)
- Sie kennen SQL-Aggregations-Operatoren und können diese anwenden: GROUP BY ROLLUP / CUBE / GROUPING SETS sowie PIVOT/UNPIVOT
- Sie kennen die Konzepte von OLAP Data Cubes
- Sie kennen Konzepte von Data Marts, Lakes, Meshes und Data Spaces

Haus- und Selbstlernaufgaben auf übernächste (!) Woche VoWo03:
Anhänge **Data Cubes**, Fallstudie Weinhandlung, Analytische Windows
Funktionen durchlesen!

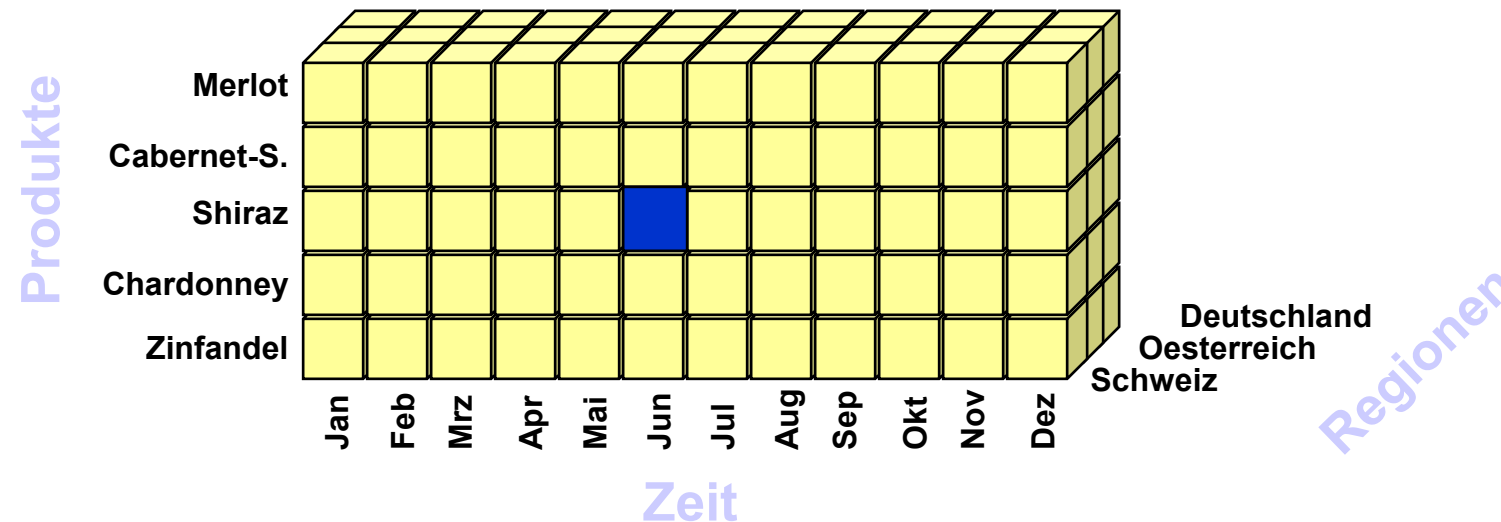
- “The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling”, by Kimball Group. 2nd Ed. John Wiley & Sons, 2002 (436 p.).
Online: <http://www.kimballgroup.com/html/articles.html>
- Bauer, A. + Günzel, H. (Hrsg.): Data Warehouse Systeme...
4. Auflage, dpunkt-Verlag, Heidelberg, 2013

ANHÄNGE

DATA CUBES

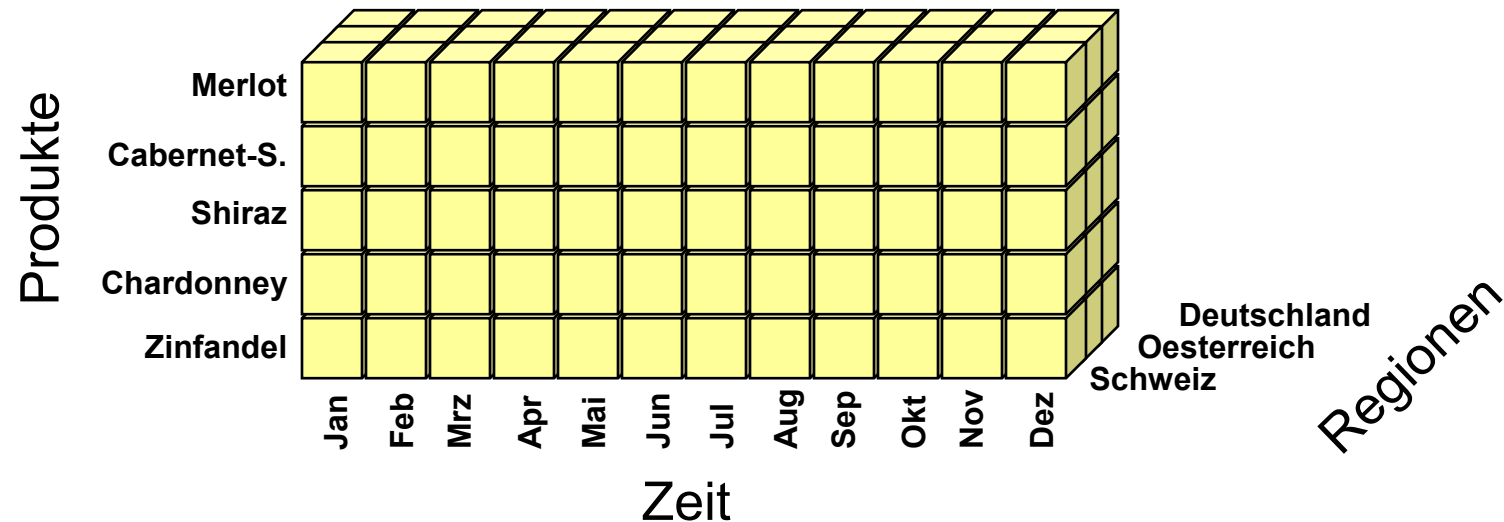
Einführung Data Cube

- Multidimensional Database - Data Cube
- Problem: Normalisierte Darstellung führt zu komplexen ineffizienten Auswertungen
- Lösung: Anordnung der Daten in einem mehrdimensionalen Würfel optimal für Auswertungen, Redundanz in den Dimensionen möglich



Einführung Data Cube ff.

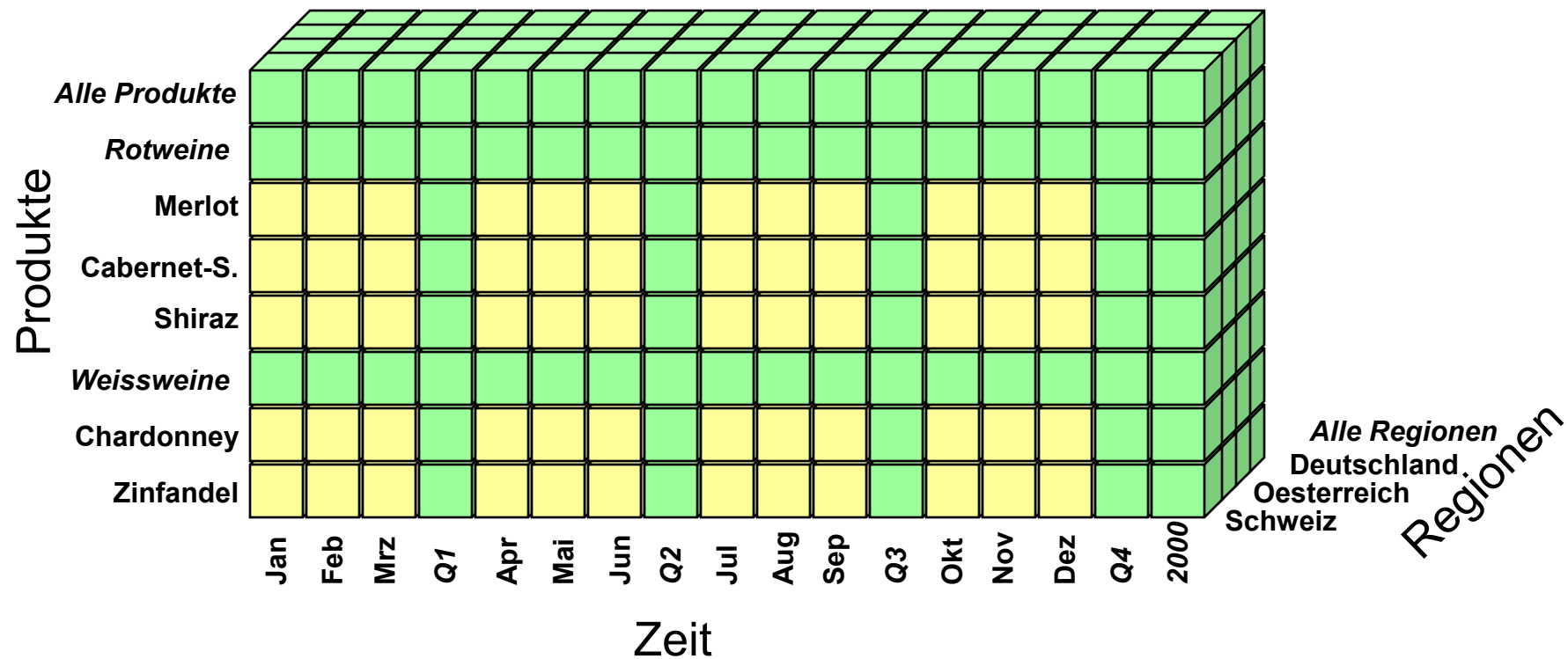
- Daten (Fakten) werden über «Dimensionen» beschrieben



- Begriffe: Multidimensionalität, Hypercubes, Ausprägungen (Members), Zellen

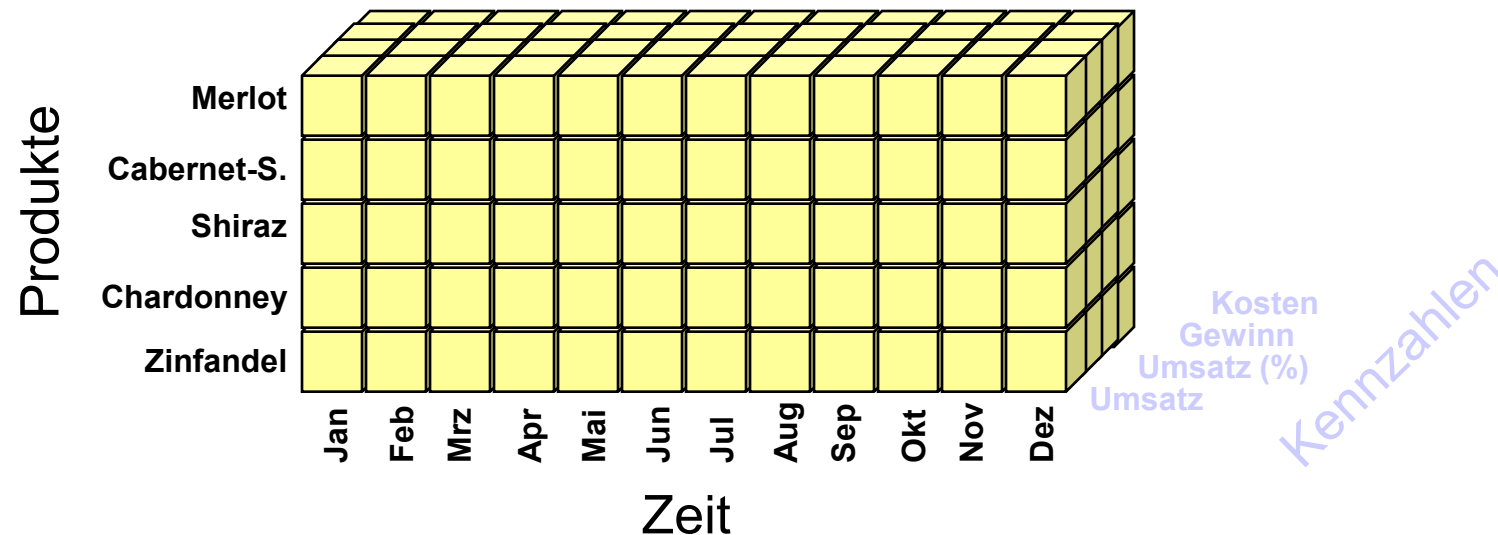
Einführung Data Cube: Hierarchien

- Dimensionen können «Hierarchien» (= «Levels») haben



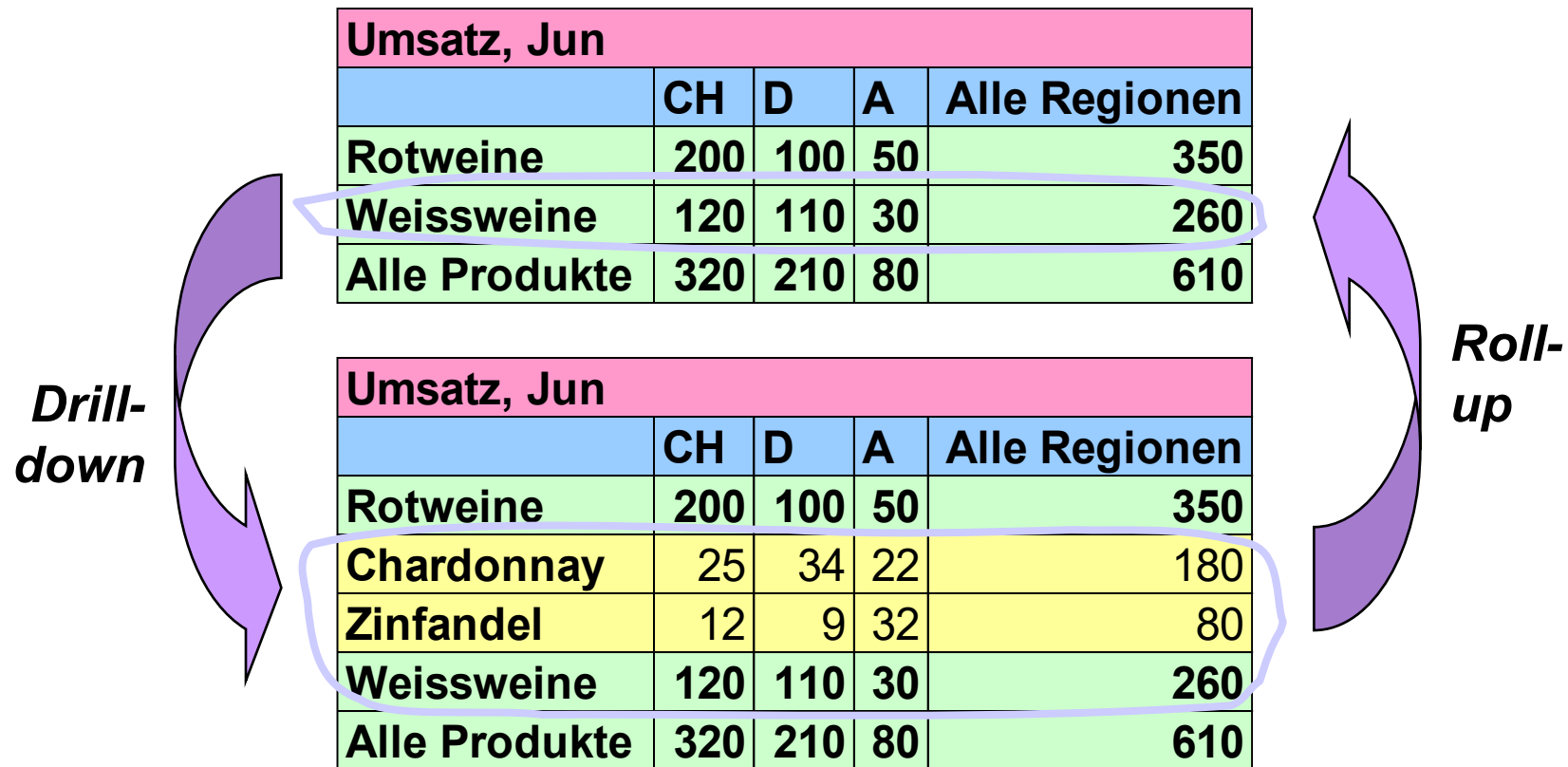
Einführung Data Cube: «Kennzahlen»-Dimension

- In der Regel werden in einem Würfel mehrere Kennzahlen angezeigt. Für die Kennzahlen/Fakten-Aspekte (Measures) wird daher meistens eine eigene Dimension geschaffen.



Einführung Data Cube: Flexible Auswertung (1)

- Die multidimensionalen Daten können am Bildschirm flexibel präsentiert werden.



Einführung Data Cube: Flexible Auswertung (2)

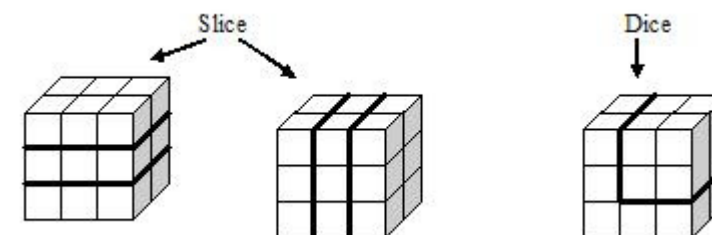
- Die multidimensionalen Daten können am Bildschirm flexibel präsentiert werden.

Kennzahlen	Umsatz	
<i>Umsatz</i>		
Gewinn		Alle Regionen
Produkte	Rotweine	3200
Regionen	Weissweine	1900
Zeit	Alle Produkte	5100

Eine beliebige Kombination von Dimensionen und Ausprägungen kann angezeigt werden.

Kennzahlen <i>Umsatz</i> <i>Gewinn</i> Produkte <i>Regionen</i> <i>Zeit</i>	Umsatz	
		Alle Regionen
	Qtr1	900
	Qtr2	1300
	Qtr3	1200
	Qtr4	1700
	2000	5100

Slice & Dice



Einführung Data Cube: Flexible Auswertung (3)

- Die multidimensionalen Daten können am Bildschirm flexibel präsentiert werden.

Umsatz					
		CH	D	A	Alle R.
Rotw.	Qtr1	200	100	50	350
Weissw.		120	110	30	260
Alle P.		320	210	80	610
Rotw.	Qtr2	180	90	50	320
...		...	...	...	...
Alle P.		270	130	80	480
Rotw.	2000	910	390	180	1480
Weissw.		370	310	190	870
Alle P.		1280	700	370	2350



Slice & Dice

Umsatz				
		Rotw.	Weissw.	Alle P.
Qtr1	CH	200	120	320
	D	100	110	210
	A	50	30	80
	Alle R.	350	260	610
Qtr2	CH	180	100	280
	D	90	50	140
	...	...	...	...
2000	CH	910	370	1280
	D	390	310	700
	A	180	190	370
	Alle R.	1480	870	2350

Die Achsen können beliebig ausgetauscht werden.

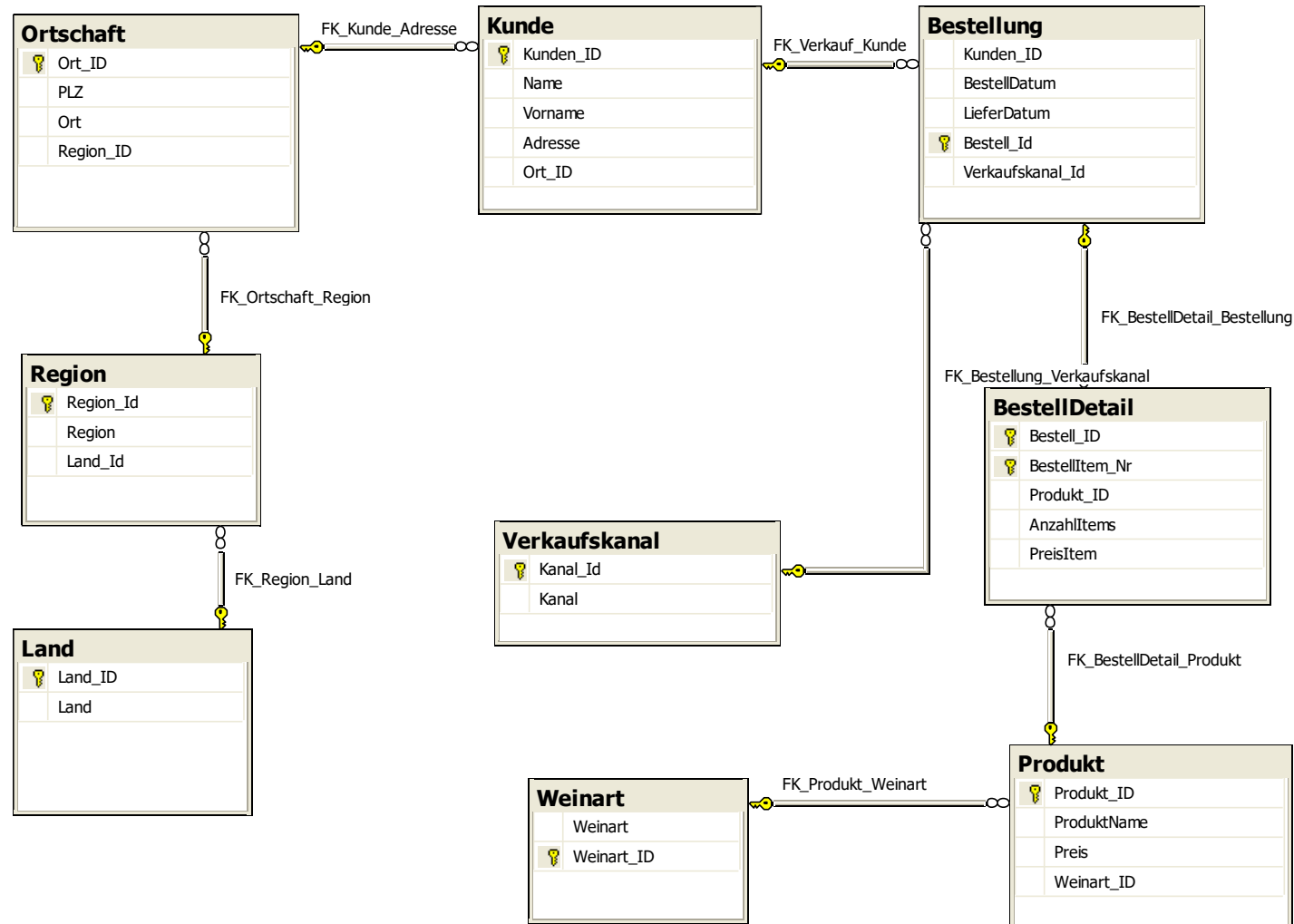
- Cube besteht aus Measures und Dimensions
- Measure (Synonym Fakt), beschreiben numerische Grössen
Measure-Group = Menge von Measures mit denselben Dimensionsverknüpfungen.
- Dimension beschreibt Measure-Groups. Eine Dimension besteht aus einem oder mehreren Attributen.
- Hierarchien: Jedes Attribut kann eine Hierarchie (All => einzelner Wert) aufweisen. Auf den Dimensionen können Hierarchien zwischen den Attributen definiert werden (für Drill-down),
- Dimensionen sind mit den Measure-Groups verknüpft
- Abfragen: Spezialisierte Sprachen, wie z.B. MDX von Microsoft

ANHANG

FALLSTUDIE WEINHANDLUNG

- Kunde bestellen per Telefon, übers Internet oder per Fax. Eine Bestellung umfasst mehrere Teilbestellungen für eine bestimmte Weinart (Anzahl, Preis)
- Jeder Wein gehört zu einer Weinart
- Zu jedem Kunde wird seine Adresse, Wohnort, Region und Land erfasst.
- Die Firma budgetiert ihre Verkaufszahlen halbjährlich pro Weinart und Land
- Auswertungen
 - Verkaufsstatistik aufgeschlüsselt nach Weinart, Wein, Land, Zeit, Kunde etc.
 - Vergleich mit Budget und mit Vorjahreszahlen

Fallstudie Weinhandlung: OLTP-Schema



DW-Design Weinhandlung

- Welche «Sterne»?
- Welche Dimensionen?
- Pro Stern
 - Welche Fakten mit welcher Granularität?

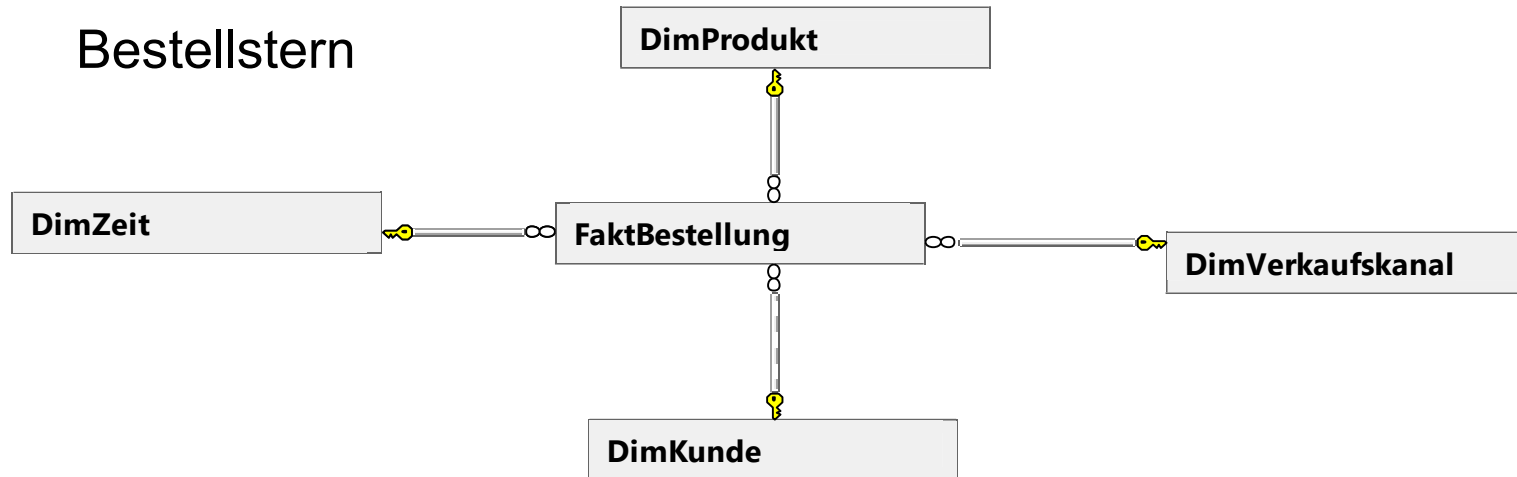
Sterne

- FaktBestellung

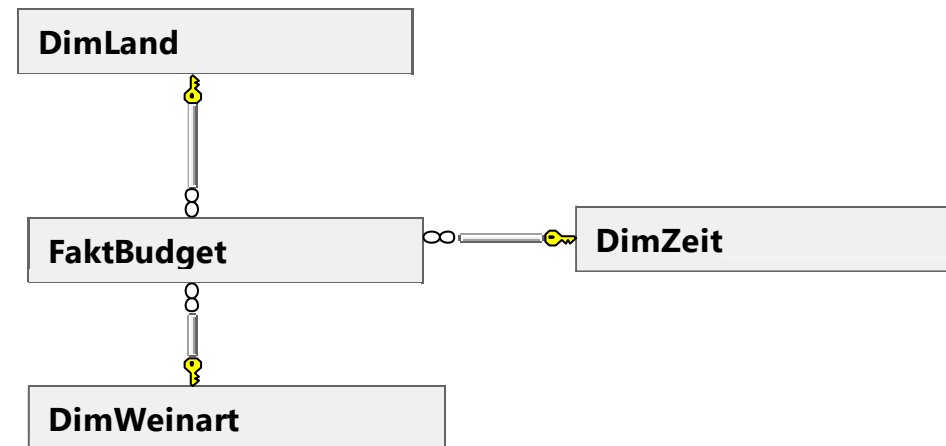
- FaktBudget

Ein erster Ansatz

Bestellstern



Budgetstern



FaktBestellung

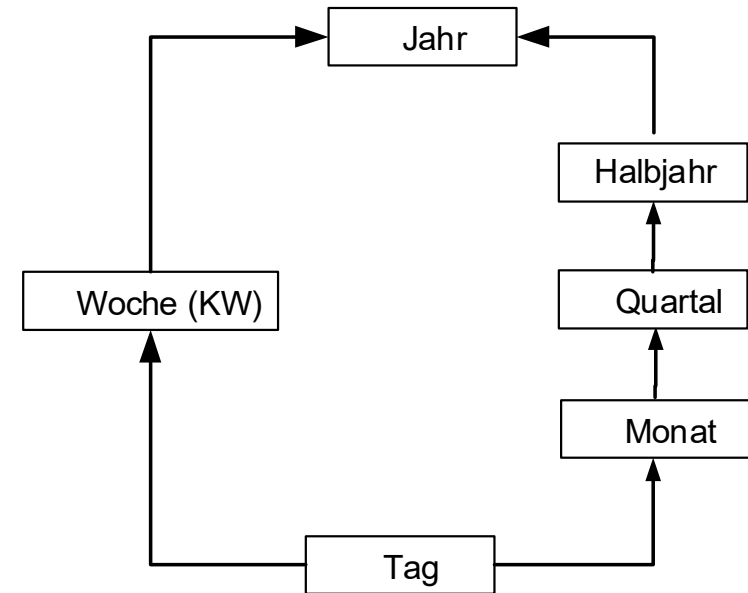
	Bestell_ID	BestellItem_Nr	Kunden_ID	BestellZeit_ID	LieferZeit_ID	Verkaufskanal_ID	Produkt_ID	AnzahlItems	PreisItem	TotPreis
1	1	1	1	1727	NULL	1	7	2	26	52.00
2	2	1	4	1728	NULL	3	2	2	19	38.00
3	3	1	5	1729	NULL	2	1008	2	32	64.00
4	4	1	1	1730	2113	1	2	2	19	38.00
5	5	1	6	1731	NULL	3	7	2	26	52.00
6	6	1	6	1731	NULL	2	2	2	19	38.00
7	7	1	6	1732	NULL	1	1008	2	32	64.00
8	8	1	6	1732	NULL	3	2	2	19	38.00
9	9	1	6	1732	NULL	2	1008	2	32	64.00

FaktBestellung *	
Bestell_ID	
BestellItem_Nr	
Kunden_ID	
BestellZeit_ID	
LieferZeit_ID	
Verkaufskanal_ID	
Produkt_ID	
AnzahlItems	
PreisItem	
TotPreis	

Granularität = Bestell-Item

- Zeit-Dimension häufig und wichtig in Datenanalyse
- Attribute haben funktionale Abhängigkeiten → Hierarchie
- KW können zu zwei Monaten gehören, daher in der Regel mehrere Hierarchien: z.B.

- Weitere Hierarchien auf der Zeitdimension
 - Fiskaljahr, Kalenderjahr
 - Verschiedene Kalenderdefinitionen (z.B. wann fängt die Woche 1 an?)



Weinhandlung Zeitdimension im DW

ZeitId	Datum	DatumString	Jahr	Monat	Tag	MonatsName	WochentagNr	Wochentag	WochenNr	Quartal	Halbjahr
1	5/1/2004 00:00:00	01.05.2004	2004	5	1	May	1	Saturday	18	Q 2 2004	H 1 2004
2	6/2/2004 00:00:00	02.06.2004	2004	6	2	June	2	Wednesday	23	Q 2 2004	H 2 2004
3	8/3/2004 00:00:00	03.08.2004	2004	8	3	August	3	Tuesday	32	Q 3 2004	H 2 2004
4	10/4/2004 00:00:00	04.10.2004	2004	10	4	October	4	Monday	41	Q 3 2004	H 2 2004
5	4/23/2004 00:00:00	23.04.2004	2004	4	23	April	23	Friday	17	Q 2 2004	H 1 2004
6	5/23/2004 00:00:00	23.05.2004	2004	5	23	May	23	Sunday	22	Q 2 2004	H 1 2004
7	3/10/2004 00:00:00	10.03.2004	2004	3	10	March	10	Wednesday	11	Q 1 2004	H 1 2004
8	6/2/2004 00:00:00	02.06.2004	2004	6	2	June	2	Wednesday	23	Q 2 2004	H 2 2004
9	7/10/2004 00:00:00	10.07.2004	2004	7	10	July	10	Saturday	28	Q 2 2004	H 2 2004
10	7/23/2004 00:00:00	23.07.2004	2004	7	23	July	23	Friday	30	Q 2 2004	H 2 2004
11	8/2/2004 00:00:00	02.08.2004	2004	8	2	August	2	Monday	32	Q 3 2004	H 2 2004
12	8/3/2004 00:00:00	03.08.2004	2004	8	3	August	3	Tuesday	32	Q 3 2004	H 2 2004

Primary Business
Key Key

(alternate

Generierung der Zeitdimension

```
BestellDatum AS Datum,  
CONVERT(varchar, BestellDatum, 104) AS DatumString,
```

```
DATEPART(day, BestellDatum) AS Tag,  
DATEPART(year, BestellDatum) AS Jahr,  
DATEPART(month, BestellDatum) AS Monat,  
DATENAME(month, BestellDatum) AS MonatsName,  
DATEPART(day, BestellDatum) AS WochentagNr,  
DATENAME(weekday, BestellDatum) AS Wochentag,  
DATENAME(week, BestellDatum) AS WochenNr,
```

```
'Q' + CAST(DATEPART(month, BestellDatum) / 4 + 1 AS varchar)  
+ '' + CAST(DATEPART(year, BestellDatum) AS VARCHAR) AS  
Quartal,
```

```
'H' + CAST(DATEPART(month, BestellDatum) / 6 + 1 AS varchar)  
+ '' + CAST(DATEPART(year, BestellDatum) AS VARCHAR) AS  
Halbjahr,
```

```
CREATE TABLE [DimZeit] (  
    [Zeit_ID] [int] PRIMARY KEY,  
    [Datum] [datetime] NULL,  
    [DatumString] [varchar](20) NULL,  
    [Jahr] [int] NULL,  
    [Monat] [int] NULL,  
    [Tag] [int] NULL,  
    [MonatsName] [varchar](10) NULL,  
    [WochentagNr] [int] NULL,  
    [Wochentag] [varchar](10) NULL,  
    [WochenNr] [int] NULL,  
    [Quartal] [varchar](10) NULL,  
    [Halbjahr] [varchar](10) NULL  
)
```

Analyse und Zerlegung der
auftretenden Datumswerte
(BestellDatum) im ETL-Prozess beim
Laden des DW (SQL-Server)

Weitere Dimension: DimKunde

DimKunde

Kunden_ID	Name	Vorname	Adresse	PLZ	Ort	OrtKuerzel	Region	Land_ID
1	Müller	Peter	Rosenweg 7	6000	Peterlingen	PET	Ostschweiz	1
4	Muster	Thomas	Fliederweg 18	50000	München	MUN	Süddeutschland	2
5	Schumacher	Michael	Maximilianstrasse 77	50000	München	MUN	Süddeutschland	2
10	Jucker	Jürg	Bodenwies	6000	Peterlingen	PET	Ostschweiz	1

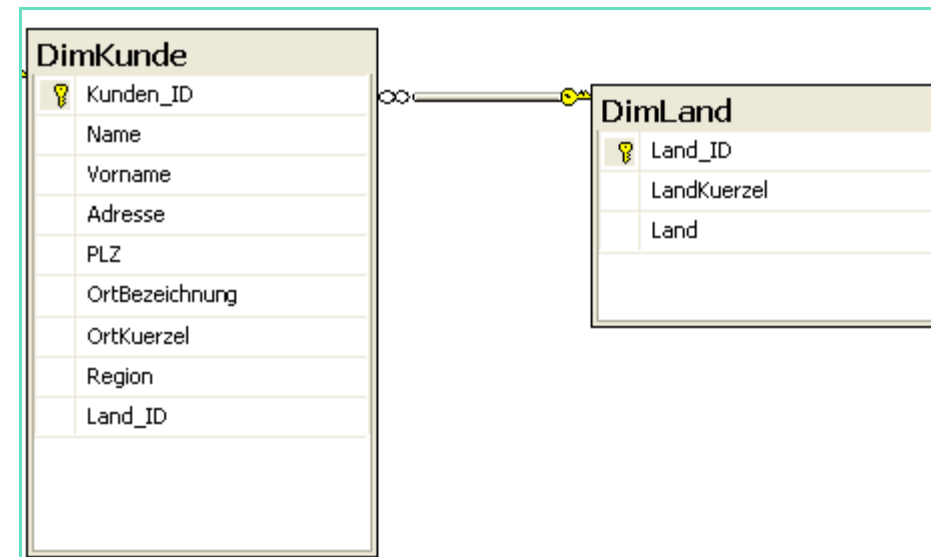
DimLand

Land_Id	LandKuerzel	Land
1	CH	Schweiz
2	D	Deutschland
3	A	Oesterreich

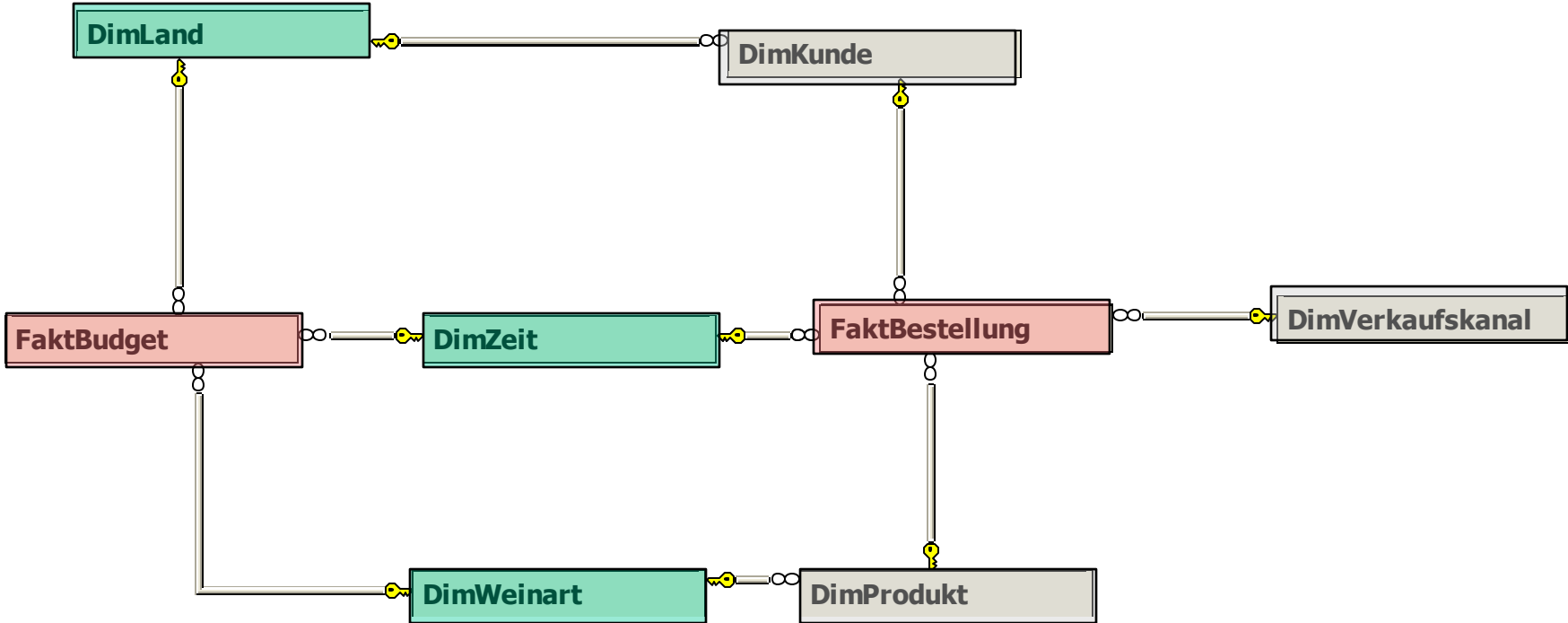
Hierarchien

Kunde → Ort → Region → Land

Wieso die Aufteilung in 2 Tabellen?



Beispiel Weinhandlung – Data Warehouse



Star- (Snowflake) Schema für das DW



Fakt-Tabelle

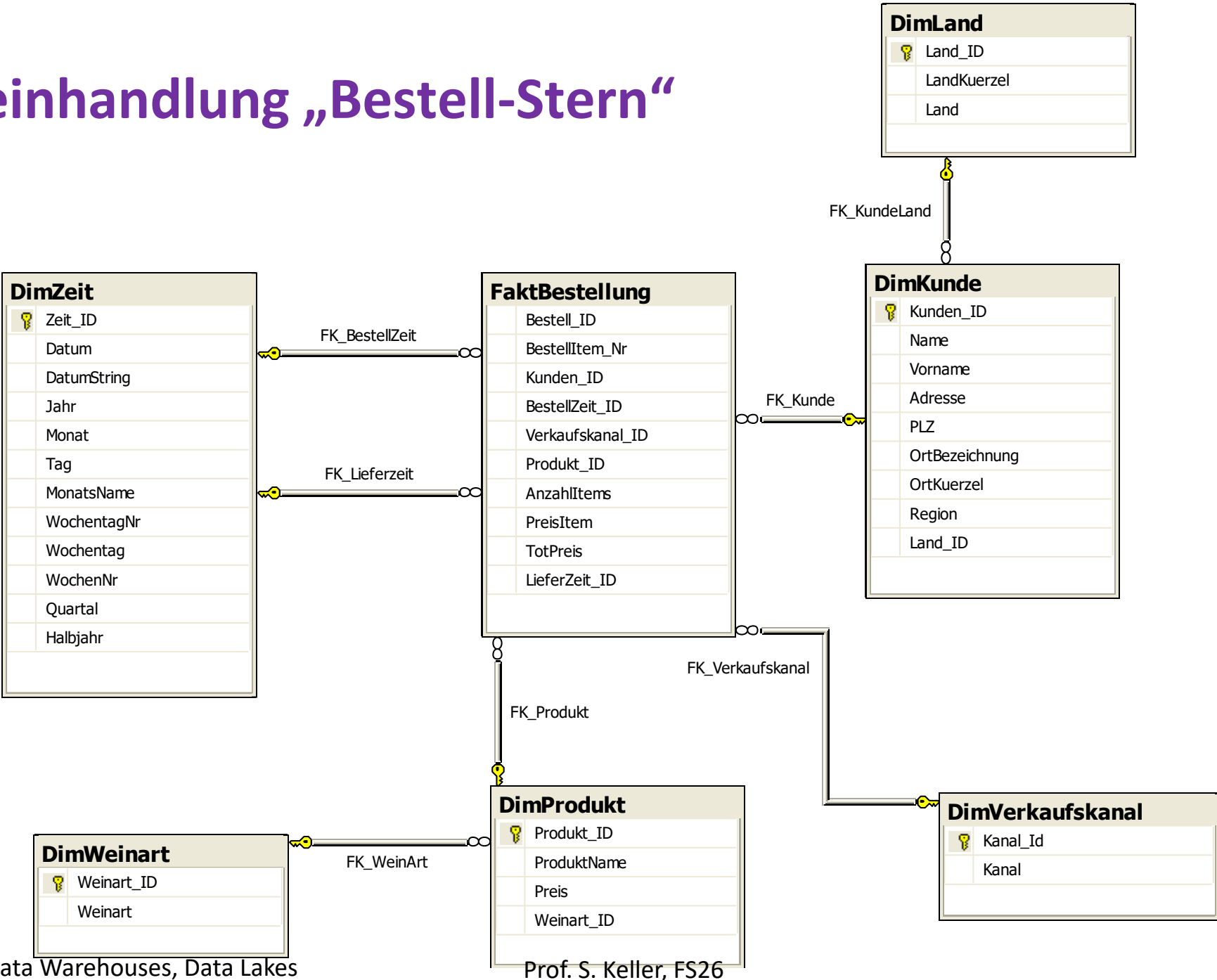


Dimension

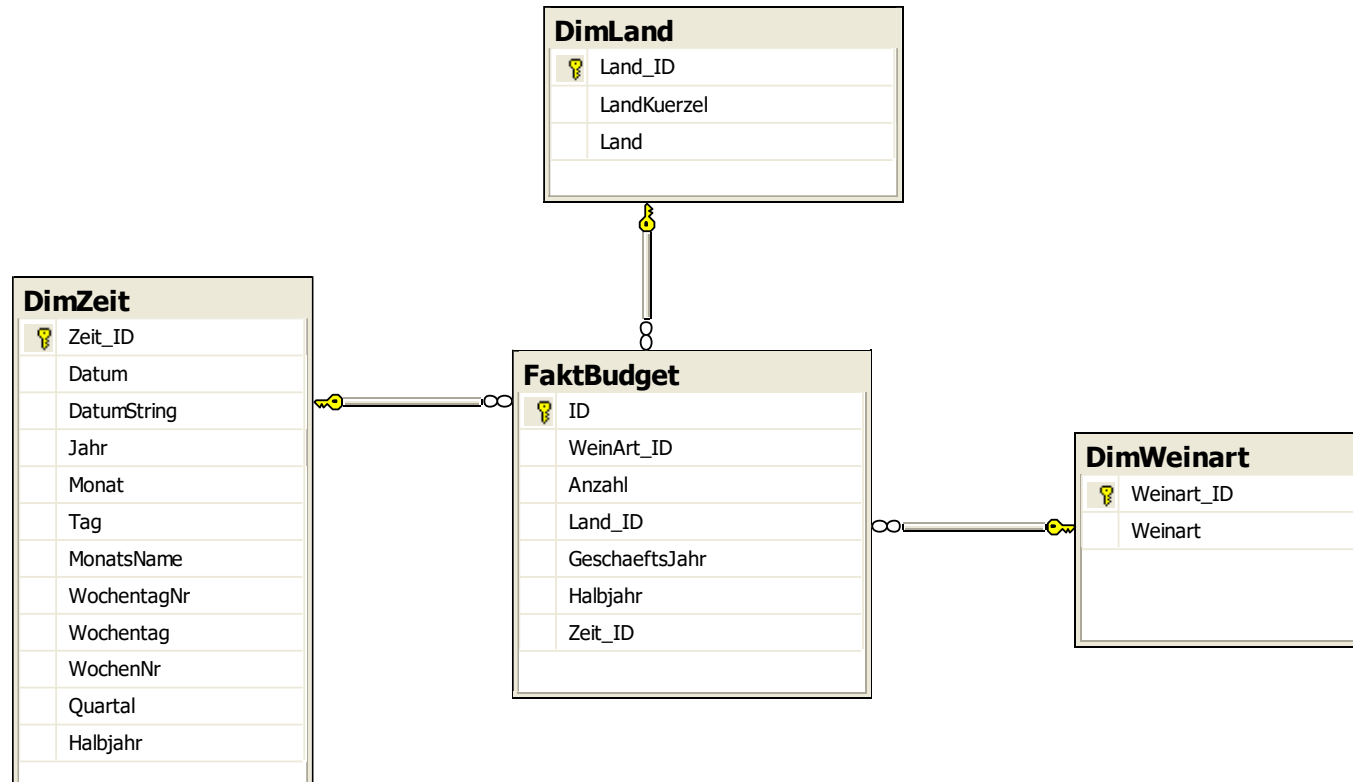


Gemeinsame Dimension

DW Weinhandlung „Bestell-Stern“



DW- Weinhandlung „Budget-Stern“



Star- (Snowflake) Schema für das DW

- relationale Speicherung eines „Würfels“
- Faktentabelle mit den auszuwertenden numerischen (=aggregierbar) Daten mit Fremdschlüsseln auf Dimensionstabellen
- Dimensionstabelle speichern ‚Metainformationen‘ zu den Fakten (in der Form von Strings)
- Dimensionstabellen weisen in der Regel einen Surrogatschlüssel auf.
- Redundanz in den Dimensionen Produkt und Zeit (Star)
- Jede Faktentabelle bildet mit den verknüpften Dimensionen einen Stern.
- Auswertungen auf den Fakten mit 1-stufigen Joins
- Abfragen zwischen verschiedenen Faktentabellen sind nur über gemeinsame Dimensionen möglich (Bsp. Vergleich aktuelle Verkaufszahlen mit Budget gruppiert über die Zeit)

Beispiel einstufiger Join

- Verkaufsstatistik pro Halbjahr

```
SELECT  zDim.Jahr, zDim.Halbjahr, prodArt.Weinart, prod.ProduktName,  
        sum(TotPreis) "Verkaufspreis", sum(AnzahlItems) "AnzItems"  
FROM FaktBestellung facts  
join DimZeit zDim on facts.BestellZeit_Id=zDim.Zeit_Id  
join DimProdukt prod on facts.Produkt_ID=prod.Produkt_ID  
join DimWeinart prodArt on prod.Weinart_ID=prodArt.Weinart_ID  
GROUP BY zDim.Jahr, zDim.Halbjahr,  
        prodArt.Weinart, prod.ProduktName
```


ANHANG - Repetition zum Nachschlagen!

ANALYTISCHE FUNKTIONEN IN SQL (WINDOW-FUNKTIONEN)

- Begriff «Analytische Funktionen»:
 - “Analytic OLAP Functions”, Synonym: “SQL Window Functions”
 - Analytische Funktionen vs. Aggregations-Funktionen
- Typische Abfragen in einem Data Warehouse
 - Verkaufszahlen pro Monat und Filiale
 - Aggregationen abgedeckt durch „group by“ und „cube“-Operator
 - Prozentuale Veränderung der Verkaufszahlen verglichen mit einer Vorperiode
 - „Moving Average“ der Verkaufszahlen über die letzten 3 Monate
 - Welches sind die 5 „besten“ Produkte im laufenden Jahr?
 - Erweiterung von SQL mit Window Queries und analytischen Funktionen

Analytische Funktionen: SQL Window-Funktionen

```
SELECT Abtnr, Name, Salaer
, ROW_NUMBER() OVER (ORDER BY Name DESC) ord
, RANK() OVER (ORDER BY Salaer DESC) rank
, DENSE_RANK() OVER (ORDER BY Salaer DESC) dense_rank
, NTILE(4) OVER (ORDER BY Salaer DESC) ntile4
FROM Angestellter
WHERE Name <= 'm%'
ORDER BY ord
```

	Abtnr	Name	Salaer	ord	rank	dense_rank	ntile4
1	3	Kramer, Luise	4000.00	1	11	10	4
2	3	Kern, Veronika	4800.00	2	6	6	2
3	2	Heiniger, Urs	4098.00	3	10	9	4
4	4	Gschwind, Fritz	5900.00	4	4	4	2
5	3	Graber, Berta	4590.00	5	7	7	3
6	4444	Delete, Me	4590.00	6	7	7	3
7	4	Danuser, Vreni	5100.00	7	5	5	2
8	3	Beeler, Hans	3780.00	8	12	11	4
9	2	Becker, Fritz	6346.00	9	3	3	1
10	3	Ammann, Fritz	7890.00	10	2	2	1
11	1	Affolter, Vreni	4123.00	11	9	8	3
12	3	Aarburg, Werner	9000.00	12	1	1	1

SQL definiert eine Vielzahl von analytischen, d.h. Window-Funktionen für die Auswertung der Daten im DW

- Jede analytische Funktion bezieht sich auf Teilmengen der Daten, auf sogenannte Partitionen. Anhand der Partitionen wird definiert, wie die Resultate der analytischen Funktion gruppiert werden. Die Aufteilung der Partitionen wird durch die Klausel PARTITION BY angegeben. Wird sie weggelassen, bezieht sich die analytische Funktion auf die gesamte Resultatmenge.
- Durch die Klausel ORDER BY kann festgelegt werden, wie die Daten innerhalb der Partition sortiert werden. Für viele der analytischen Funktionen ist die Angabe der Reihenfolge zwingend, da sie für die Funktionsberechnung wesentlich ist.
- Die Reihenfolge der Ausgabe der Resultatmenge kann jedoch von der Reihenfolge der Berechnung abweichen und wird mit einem ORDER BY am Ende des SELECT-Befehls definiert.
- Falls für eine analytische Funktion ein ORDER BY spezifiziert wurde, kann die Menge der für die Berechnung relevanten Datensätze durch die Definition eines Windows zusätzlich eingeschränkt werden. Normalerweise umfasst das Window die Datensätze vom Beginn der Partition bis zum aktuellen Datensatz, es kann aber auch explizit durch ROWS (physische Grenze) oder RANGE (logische Grenze) eingeschränkt werden.

Analytische Funktionen: SQL Window-Fn. mit PARTITION BY

```
SELECT Abtnr, Name, Salaer
, SUM(Salaer) OVER (PARTITION BY abtnr) SalaerAbt
, RANK() OVER (ORDER BY Salaer DESC) Rang
, RANK() OVER (PARTITION BY abtnr ORDER BY Salaer DESC) RangInAbt
FROM angestellter
WHERE AbtNr IN (1,2)
ORDER BY AbtNr, RangInAbt
```

	ABTNR	NAME	SALAER	SALAERABT	RANG	RANGINABT
1	1	Marxer, Markus	10580	43465	2	1
2	1	Zuercher, Hedi	10019	43465	3	2
3	1	Meier, Franz	9765	43465	4	3
4	1	Wernli, Peter	8978	43465	5	4
5	1	Affolter, Vreni	4123	43465	7	5
6	2	Widmer, Anna	12010	22454	1	1
7	2	Becker, Fritz	6346	22454	6	2
8	2	Heiniger, Urs	4098	22454	8	3

Partition by kann in Kombination mit gewissen Aggregatfunktionen verwendet werden
Partition By zerlegt die Tupels in eine Menge (ähnlich wie GROUP BY)
(mehr Beispiele in den Uebungen)
(Unterschied: Group By gibt ein Resultattupel pro Partition, Partition BY hingegen ein Resultattupel für jedes Tupel in der Partition)

Bsp: Verkaufsstatistik pro Halbjahr mit Rollup

```
SELECT zDim.Jahr, zDim.Halbjahr, prodArt.Weinart, prod.ProduktName,  
       sum(TotPreis) "Verkaufspreis", sum(AnzahlItems) "AnzItems"  
FROM FaktBestellung facts  
JOIN DimZeit zDim ON facts.BestellZeit_Id=zDim.Zeit_Id  
JOIN DimProdukt prod ON facts.Produkt_ID=prod.Produkt_ID  
JOIN DimWeinart prodArt ON prod.Weinart_ID=prodArt.Weinart_ID  
GROUP BY ROLLUP(zDim.Jahr,  
                zDim.Halbjahr,  
                prodArt.Weinart,  
                prod.ProduktName )
```

	Jahr	Halbjahr	Weinart	ProduktName	Verkaufspreis	AnzItems
81	2007	H1 2007	Weisswein	Shiraz	6216.00	296
82	2007	H1 2007	Weisswein	NULL	30630.00	1076
83	2007	H1 2007	NULL	NULL	141441.00	6429
84	2007	H2 2007	Rotwein	Chardonnay	129577.00	5603
85	2007	H2 2007	Rotwein	Merlot	41312.00	2582
86	2007	H2 2007	Rotwein	Zinfandel	14250.00	750
87	2007	H2 2007	Rotwein	NULL	185139.00	8935
88	2007	H2 2007	Weisswein	Cabernet-S.	36588.00	1170
89	2007	H2 2007	Weisswein	Shiraz	11004.00	524
90	2007	H2 2007	Weisswein	NULL	47592.00	1694
91	2007	H2 2007	NULL	NULL	232731.00	10629
92	2007	NULL	NULL	NULL	374172.00	17058
93	2008	H1 2008	Rotwein	Chardonnay	66227.00	2875
94	2008	H1 2008	Rotwein	Merlot	21888.00	1368
95	2008	H1 2008	Rotwein	Zinfandel	3401.00	179
96	2008	H1 2008	Rotwein	NULL	91516.00	4422
97	2008	H1 2008	Weisswein	Cabernet-S.	21988.00	702
98	2008	H1 2008	Weisswein	Shiraz	6090.00	290
99	2008	H1 2008	Weisswein	NULL	28078.00	992
100	2008	H1 2008	NULL	NULL	119594.00	5414
101	2008	NULL	NULL	NULL	119594.00	5414
102	NU...	NULL	NULL	NULL	1595968.00	71360

YEAR-To-Date auf Level Quartal

```
SELECT t.Jahr, t.Quartal, SUM(s.TotPreis) AS Betrag,  
SUM(SUM(s.TotPreis)) OVER (PARTITION BY t.Jahr ORDER BY t.Quartal) YTD  
FROM FaktBestellung s  
JOIN DimZeit t ON s.Bestellzeit_ID = t.Zeit_ID  
GROUP BY t.Jahr, t.Quartal;
```

Jahr	Quartal	BETRAG	YTD
2002	Q1 2002	2114	2114
2002	Q2 2002	10170	12284
2002	Q3 2002	11486	23770
2002	Q4 2002	2342	26112
2003	Q1 2003	2152	2152
2003	Q2 2003	13018	15170
2003	Q3 2003	19256	34426
2003	Q4 2003	8614	43040
2004	Q1 2004	4200	4200
2004	Q2 2004	15288	19488
Es sind mehr als 10 Zeilen verfügbar. Erhöhen Sie den Zeilen-Selector, um weitere Zeilen anzuzeigen.			

Top-3 Kunden nach Verkaufszahlen

```
with Verkaufsrang AS (  
  select k.name, sum(f.tottpreis) Summe,  
         rank() over (order by sum(f.tottpreis) desc) as rang  
  from faktbestellung f  
  inner join DimKunde k on f.kunden_id=k.kunden_id  
  group by k.name  
)  
select * from Verkaufsrang  
where rang <=3  
order by 2 desc;
```

	RANK	NAME	SUMME	RANK
1		Muster	4856320	1
2		Meier	417792	2
3		Schumacher	361216	3

Analytische Funktionen: SQL Window-Fn. mit RANGE...

```
SELECT
  abtnr,
  name,
  salaer,
  COUNT(*) OVER (PARTITION BY abtnr ORDER BY salaer
                  RANGE BETWEEN 500 PRECEDING AND 500 FOLLOWING) anz_sal
FROM Angestellter
ORDER BY salaer;
```

RZ	ABTNR	RZ	NAME	RZ	SALAER	RZ	ANZ_SAL
	1		Affolter, Vreni		4124		1
	1		Wernli, Peter		8979		1
	1		Meier, Franz		9766		2
	1		Zuercher, Hedi		10020		2
	1		Marxer, Markus		10581		1
	2		Heiniger, Urs		4099		1
	2		Becker, Fritz		6347		1
	2		Widmer, Anna		12011		1
	3		Beeler, Hans		3781		2
	3		Kramer, Luise		4001		2
	3		Graber, Berta		4591		3
	3		Kern, Veronika		4801		3
	3		Pauli, Monika		5090		3
	3		Wehrli, Anton		5981		1
	3		Widmer, Karl		7501		2

Current Row

Partition

Window

„Moving Average“ über die letzten 3 Monate mit Windowing-Funktion

```
SELECT
t.MonatsName, sum(s.TotPreis) AS AmountPerMonth,
ROUND(AVG(sum(s.TotPreis)) OVER
  ( ORDER BY t.MonatsName rows between 2 preceding and current row),0)
AS MovingAverage
FROM FaktBestellung s
INNER JOIN DimZeit t ON s.BestellZeit_ID = t.Zeit_ID
GROUP BY t.MonatsName
ORDER BY t.MonatsName;
```

	R Z	MONATSNAME	R Z	AMOUNTPERMONTH	R Z	MOVINGAVERAGE
1		2002-01		208		208
2		2002-02		3584		1896
3		2002-03		4664		2819
4		2002-04		10112		6120
5		2002-05		14568		9781
6		2002-06		16000		13560
7		2002-07		21924		17497
8		2002-08		9368		15764
9		2002-09		14652		15315
10		2002-10		4352		9457
11		2002-11		3876		7627
12		2002-12		1140		3123
13		2003-01		208		1741
14		2003-02		5760		2369
15		2003-03		2640		2869
16		2003-04		15708		8036

Analytische Funktionen: SQL Window-Fn. LAG und LEAD

```
SELECT t.MonatsName, sum(s.TotPreis) AS this_month,  
       LAG(sum(s.TotPreis),3) OVER ( ORDER BY t.MonatsName)  
                                     AS three_months_ago  
FROM FaktBestellung s  
INNER JOIN DIMZeit t on s.BestellZeit_ID=t.Zeit_ID  
GROUP BY t.MonatsName;
```

	MONATSNAME	THIS_MONTH	THREE_MONTHS_AGO
1	2002-01	208	(null)
2	2002-02	3584	(null)
3	2002-03	4664	(null)
4	2002-04	10112	208
5	2002-05	14568	3584
6	2002-06	16000	4664
7	2002-07	21924	10112

Analytische Funktionen: Weitere SQL Window-Funktionen

- Operator (Funktion)
«GROUPING(<col1, col2>)»:
 - Gibt ein Integer zurück, der eine «Bitmaske» codiert.
 - Zeigt an, welche Argumente (Kolonnen/Attribute) *nicht* in den aktuellen Gruppierungssatz aufgenommen werden.
 - PostgreSQL-Dokumentation:
<https://www.postgresql.org/docs/current/functions-aggregate.html#FUNCTIONS-GROUPING-TABLE> .
 - Beispiel →

```
SELECT modell, jahr, sum(anzahl) "anz",  
grouping(modell, jahr) "g"  
FROM autoverkaeufe  
GROUP BY ROLLUP (modell, jahr)  
ORDER BY modell, jahr;
```

modell	jahr	anz	g
-----	-----	-----	---
Ford	1990	189	0
Ford	1991	116	0
Ford	1992	128	0
Ford	(null)	433	1
Opel	1990	154	0
Opel	1991	198	0
Opel	1992	156	0
Opel	(null)	508	1
(null)	(null)	941	3